

Manual de Referência: biblioteca PySpectral Quantum Solver

Vagner Jandre Monteiro

IPRJ/UERJ – PPGMC/DO

Resumo:

A biblioteca PySpectral foi desenvolvida a partir de uma versão pré-existente, originalmente implementada nas linguagens MAPE e OCTAVE. Nesta nova versão, foi realizada uma releitura dos métodos já existentes e adicionadas novas funcionalidades, como o cálculo do momento da partícula e um método para otimização do comprimento de análise do problema.

Apesar desses novos recursos, o objetivo central foi atualizar a linguagem utilizada pela biblioteca, migrando-a para uma plataforma com número crescente de usuários — no caso, a linguagem Python. Python é uma linguagem acessível, cada vez mais difundida, que oferece novas ferramentas e recursos para profissionais de diversas áreas da ciência.

Nesta versão, a biblioteca Espectral foi estruturada como uma classe. Assim, ao definir um problema, o usuário cria uma instância dessa classe e utiliza os diferentes métodos disponíveis. Essa abordagem permite definir, simultaneamente, diferentes problemas e, em seguida, aplicar métodos para comparar os resultados obtidos.

O objetivo deste documento é orientar o usuário quanto aos recursos disponíveis na biblioteca e ao seu uso. Na primeira parte, são apresentados os conceitos pertinentes, as funções, parâmetros, exemplos de aplicação individual e observações relevantes. Na segunda parte, são explorados dois exemplos completos, demonstrando de forma prática a utilização de todos os recursos descritos.

Conceitos Preliminares

Em 1925, o físico austríaco Erwin Schroedinger (1887-1961) apresentou a equação que descreve a dinâmica de sistemas quânticos, como átomos, moléculas e partículas subatômica. Esta equação permite prever as propriedades físicas desses sistemas, como energias permitidas, estados estacionários e transições entre estados. Em sua forma unidimensional, a equação é escrita como:

$$\hat{H}\psi(x, t) = i\hbar \frac{\partial}{\partial t}\psi(x, t) \quad (1)$$

Em que $\hat{H}$ é o operador Hamiltoniano representando a energia total do sistema, $\hbar$ é a constante reduzida de Planck, relacionada a quantidade de movimento de uma partícula e ao seu comprimento de onda associado. A função $\psi(x, t)$ é a função de onda do sistema, uma interpretação ondulatória para um sistema com comprimento de onda de De Broglie associado. Posteriormente, o físico alemão Max Bohr (1882-1970) propôs que qualquer partícula poderia ser descrita por uma função de onda e esta função poderia descrever a probabilidade de encontrá-la em uma determinada posição em um dado instante de tempo, levando ao conceito de densidade de probabilidade:

$$P = |\psi(x, t)|^2 \quad (2)$$

Existem poucas soluções analíticas conhecidas para esta equação, como exemplo: Partícula livre, Poço de potencial infinito, oscilador harmônico e átomo de hidrogênio. Sendo então exploradas soluções com abordagens numéricas.

Para uma partícula de massa m sob ação de um potencial $V(x)$, o operador Hamiltoniano pode ser escrito na forma:

$$\hat{H} = \frac{1}{2m} \hat{p}_x^2 + \hat{V}(x) \quad (3)$$

Na mecânica quântica, os operadores momento são elevados ao grau de operadores diferenciais, i. e. $\hat{p}_x \rightarrow i\hbar \frac{\partial}{\partial x}$. Dessa forma, a equação de Schroedinger (1) assume a forma:

$$-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} \psi(x, t) + V(x)\psi(x, t) = i\hbar \frac{\partial}{\partial t} \psi(x, t) \quad (4)$$

Esta equação é conhecida como a equação de Schroedinger dependente do tempo, uma vez que a função de onda depende da posição x e do tempo t . Utilizando o método de separação de variáveis, podemos supor que a solução do sistema é composta como o produto de duas funções independentes na sua variável, ou seja,

$$\psi(x, t) = \psi(x)\eta(t) \quad (5)$$

Após aplicação do método na Eq.(4), e considerando a constante da separação E , obtém-se uma solução analítica para a função $\eta(t)$. Dessa forma um formato possível para a solução do sistema seria:

$$\psi(x, t) = \psi(x)e^{-\frac{i}{\hbar}Et} \quad (6)$$

Substituindo o resultado apresentado na Eq. (6) na Eq. (4), obtemos a equação independente do tempo:

$$-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} \psi(x) + V(x)\psi(x) = E\psi(x) \quad (7)$$

A abordagem numérica apresentada aqui, utilizara este formato de equação para buscar soluções.

O Método Espectral

O Método de Galerkin, na forma como é conhecido atualmente, foi formalizado por Boris Grigoryevich Galerkin em 1915, em um artigo seminal publicado na revista russa *Vestnik Inzhenerov* [1]. Nesse trabalho, Galerkin propôs um procedimento geral para a resolução de equações diferenciais parciais, utilizando funções de base para projetar a equação diferencial em um espaço de dimensão finita.

A ideia central consistia em adotar funções de teste idênticas às funções de base, de modo que o resíduo da equação fosse ortogonal ao subespaço gerado por essas funções [2]. Essa formulação garantiu consistência matemática e abriu caminho para aplicações em diferentes áreas da engenharia.

O estudo de Galerkin foi motivado por problemas práticos, como a análise de estruturas elásticas e placas. Sua abordagem destacou-se pela generalidade, permitindo aplicação a uma ampla gama de problemas, tanto lineares quanto não lineares. Diferentemente de outros métodos da época, que dependiam da existência de um funcional variacional, o Método de Galerkin podia ser aplicado diretamente à forma forte da equação diferencial, tornando-se mais versátil e abrangente [3] [4].

A partir da Eq. (7), consideramos equações do tipo Schrödinger no seguinte formato:

$$-\frac{\partial^2}{\partial x^2}\psi(x) + f(x)\psi(x) = Eg(x)\psi(x) \quad (8)$$

Usando o sistema natural de unidades $\hbar = 1$.

Na Eq. (8), com $x \in [0, \infty)$, em que $f(x) = 2mV(x)$ e $g(x)$ são funções conhecidas da variável espacial x , $\psi(x)$ é a parte espacial da função de onda de um sistema quântico, e E é o autovalor de energia. Pode-se observar que a equação de Schrödinger, Eq. (7), é obtida como um caso especial, deixando $g(x) = 1$. Além disso, restringindo o domínio para $x \in [0, L]$, o equivalente a colocar duas barreiras impenetráveis infinitas em $x = 0$ e $x = L$, impondo assim as condições de contorno.

O produto interno de duas funções, definido por:

$$(\psi, \phi) = \int_0^\infty \psi^*(x)w(x)\phi(x)dx \quad (9)$$

Envolve uma função de peso adequada $w(x)$ que deve ser escolhida de modo que o operador hamiltoniano

$$\hat{H} = -\frac{d^2}{dx^2} + f(x) \quad (10)$$

seja autoadjunto, ou seja,

$$(\phi, \hat{H}\psi) = (\phi, \hat{H}\psi) \quad (11)$$

Em outras palavras, um operador linear A no espaço de Hilbert é dito autoadjunto se e somente se $A = A^*$, ou seja, se e somente se A for simétrico e $D(A) = D(A^*)$.

A biblioteca *spectral.py*, utiliza o método espectral de Galerkin. Neste método, a solução $\psi(x)$ é aproximada por uma expansão finita de funções de base $\phi_k(x)$:

$$\psi(x) \approx \psi_N(x) = \sum_{n=1}^N c_n \phi_n(x) \quad (12)$$

Em que c_n são os coeficientes a serem determinados e N é o número de funções de base.

A essência do método de Galerkin é impor que o resíduo (o erro ao substituir a solução aproximada na equação diferencial) seja ortogonal a cada uma das funções de base. Essa condição é formulada como um produto interno, de acordo com a Eq. (9), que, no caso da Equação de Schrödinger, resulta em um problema de autovalor generalizado de matriz:

$$\Delta A = E \Gamma A \quad (13)$$

Em que A é o vetor de coeficientes desconhecidos, E é o autovalor (a energia), e as matrizes Δ e Γ são construídas a partir das integrais que representam os termos de energia cinética, potencial e sobreposição, respectivamente.

Para problemas com condições de contorno de Dirichlet (função de onda zero nas fronteiras, como na condição aqui abordada), a base de funções seno são uma escolha conveniente, pois satisfazem naturalmente as condições de contorno $\psi(0) = \psi(L) = 0$. Dessa forma, o método consiste em considerar uma autofunção ψ_p associada ao p -ésimo autovalor E_p . Escolhamos uma base ortonormal finita de N funções para expandirmos $\psi_p(x)$. Uma possível escolha é:

$$\psi_p(x) \approx \sqrt{\frac{2}{L}} \sum_{n=1}^N A_n^{(p)} \sin\left(\frac{n\pi x}{L}\right) \quad (14)$$

Em que $A_n^{(p)}$ são coeficiente a determinar. O método de Galerkin requer que as funções da base escolhida satisfaçam as condições de contorno em $x = 0$ e $x = L$. De maneira análoga, expandindo os termos contendo $f(x)$ e $g(x)$ na Eq. (8),

$$f(x)\psi_p(x) \approx \sqrt{\frac{2}{L}} \sum_{n=1}^N F_n^{(p)} \sin\left(\frac{n\pi x}{L}\right) \quad (15)$$

$$g(x)\psi_p(x) \approx \sqrt{\frac{2}{L}} \sum_{n=1}^N G_n^{(p)} \sin\left(\frac{n\pi x}{L}\right) \quad (16)$$

O que nos leva as seguintes relações para os coeficientes:

$$F_n^{(p)} \approx \sum_{m=1}^N \Phi_{n,m} A_m^{(p)}, \quad (17)$$

$$G_n^{(p)} \approx \sum_{m=1}^N \Gamma_{n,m} A_m^{(p)}, \quad (18)$$

de maneira que:

$$\Phi_{n,m} = \frac{2}{L} \int_0^L \sin\left(\frac{n\pi x}{L}\right) f(x) \sin\left(\frac{m\pi x}{L}\right) dx, \quad (19)$$

$$\Gamma_{n,m} = \frac{2}{L} \int_0^L \sin\left(\frac{n\pi x}{L}\right) g(x) \sin\left(\frac{m\pi x}{L}\right) dx. \quad (20)$$

Levando estas expansões na Eq. (8), obtemos a seguinte equação:

$$\left(\frac{n\pi}{L}\right)^2 A_n^{(p)} + \sum_{m=1}^N \Phi_{n,m} A_m^{(p)} = E \sum_{m=1}^N \Gamma_{n,m} A_m^{(p)}, \quad (21)$$

Que pode ser expressa na forma matricial:

$$\Delta A^{(p)} = E_p \Gamma A^{(p)}, \quad (22)$$

em que os elementos da matriz Γ ($N \times N$) são dados pela Eq. (20), $A^{(p)}$ é uma matriz coluna ($1 \times N$) com elementos $A_m^{(p)}$ e Δ é a matriz ($N \times N$) com elementos

$$\Delta_{n,m} = \left(\frac{n\pi}{L}\right)^2 \delta_{n,m} + \Phi_{n,m}. \quad (23)$$

A solução da Eq. (22) fornece os autovalores E_p e autovetores $A^{(p)}$ e assim as autofunções aproximadas $\psi_p(x)$ da Eq. (14).

EQUAÇÕES DIFERENCIAIS DE AUTOVALORES

Tratamos de equações diferenciais da forma:

$$\left(-\frac{d^2}{dx^2} + f(x)\right)\psi(x) = E g(x)\psi(x), \quad x \in [0, \infty)$$

em que $f(x)$ e $g(x)$ são funções contínuas conhecidas da variável espacial x , $\psi(x)$ é a parte espacial da função de onda de um sistema quântico, e E é o autovalor de energia.

A equação de Schrödinger é obtida como caso particular, tomando $g(x) = 1$. Restringir o domínio a $x \in [0, L]$ equivale a impor barreiras infinitas impenetráveis em $x = 0$ e $x = L$, com condições de contorno $\psi(0) = \psi(L) = 0$.

Uma autofunção aproximada $\psi_p(x)$, correspondente ao nível de energia E_p , pode ser expandida na base ortonormal de funções seno:

$$\psi_p(x) \approx \sum_{n=1}^N A_n(p) \sqrt{\frac{2}{L}} \sin\left(\frac{n\pi x}{L}\right),$$

em que os coeficientes $A_n(p)$ devem ser determinados, e um número finito N de funções base é escolhido.

Produto interno e equação matricial

O produto interno entre funções é definido como:

$$(\Psi, \Phi) = \int_0^\infty \Psi^*(x) w(x) \Phi(x) dx,$$

em que a função peso $w(x)$ é escolhida de modo que o operador Hamiltoniano seja auto-adjunto.

A combinação da equação diferencial de autovalores e da expansão acima leva à equação matricial:

$$\Delta A(p) = E_p \Gamma A(p),$$

onde $A(p)$ é um vetor coluna $1 \times N$ com elementos $A_m(p)$. A matriz Δ é quadrada $N \times N$, com elementos:

$$\Delta_{n,m} = \left(\frac{n\pi}{L}\right)^2 \delta_{n,m} + \Phi_{n,m},$$

sendo

$$\Phi_{n,m} = \frac{2}{L} \int_0^L \sin\left(\frac{n\pi x}{L}\right) f(x) \sin\left(\frac{m\pi x}{L}\right) dx,$$

e Γ é a matriz $N \times N$ com elementos:

$$\Gamma_{n,m} = \frac{2}{L} \int_0^L \sin\left(\frac{n\pi x}{L}\right) g(x) \sin\left(\frac{m\pi x}{L}\right) dx.$$

O produto interno, combinado com a expansão, leva à definição do produto escalar:

$$A(p) \cdot A(q) \equiv \sum_{n=1}^N \sum_{m=1}^N A_n(p) W_{n,m} A_m(q),$$

em que a matriz de pesos W é obtida a partir da função peso $w(x)$:

$$W_{n,m} \equiv \frac{2}{L} \int_0^L \sin\left(\frac{n\pi x}{L}\right) w(x) \sin\left(\frac{m\pi x}{L}\right) dx.$$

PRIMEIROS PASSOS COM A BIBLIOTECA PYSPECTRAL QUANTUM SOLVER

A biblioteca PySpectral foi atualizada e estruturada como uma classe na linguagem Python. Essa abordagem orientada a objetos permite que o usuário defina simultaneamente diferentes problemas e utilize diversos métodos para análise. Abaixo estão as etapas fundamentais para o seu uso:

1. Formato de Arquivo Adequado

Por ser desenvolvida em Python, os formatos de arquivo recomendados para trabalhar com a biblioteca são scripts de Python padrão (arquivos com extensão .py) ou ambientes interativos, como o Jupyter Notebook (arquivos .ipynb). O Jupyter Notebook é particularmente adequado, pois facilita a visualização imediata dos resultados impressos no console e dos gráficos gerados.

2. Importando a Biblioteca

O primeiro passo no seu código deve ser a importação da classe principal contida no módulo da biblioteca.

```
# importa as funcionalidades da biblioteca a partir de um arquivo local
from PySpectral import *
```

O arquivo pode ser encontrado e baixado através do link: <https://github.com/Legeandre/PySpectral-Quantum-Solver>.

3. Definindo o Problema Físico

Antes de criar o objeto, o usuário deve definir as funções matemáticas que descrevem o sistema quântico: o potencial $f(x)$, a função $g(x)$ e a função peso $w(x)$.

Aviso: É importante que o nome dado a essas funções não seja igual a possíveis constantes já declaradas no código, evitando ambiguidades no momento da chamada.

```
# Definição da função potencial f(x)
def f(x):
    return x**2

# Definição da função g(x)
def g(x):
    return 1

# Definição da função peso w(x)
def w(x):
    return 1
```

Nota: Caso as funções não sejam referenciadas na criação do objeto, a biblioteca assumirá automaticamente o comportamento padrão: $f = 0$, $g = 1$ e $w = 1$.

4. Criando o Objeto (Instanciação)

Ao definir um problema, o usuário deve criar uma instância da classe `SpectralMethod`, fornecendo os parâmetros físicos e de configuração do sistema.

```
# Inicia o objeto 'p'
p = SpectralMethod(
    num_levels=100,      # Número de níveis (autofunções) a serem calculados
    length=10.0,         # Comprimento do intervalo (L); deve ser positivo
    f_function=f,        # Função f(x) (potencial)
    g_function=g,        # Função g(x) associada
    weight=w,            # Função peso w(x)
    num_digits=15,       # Precisão numérica dos cálculos
    label="HalfOscillator", # Identificador do projeto (usado em títulos e logs)
    optimizer_L='y'      # 'y' para otimizar o comprimento L; caso contrário,
                        # usa o definido no atributo length.
)
```

Nota sobre o número de dígitos (num_digits): O limite de precisão nativo da linguagem Python (ponto flutuante padrão) é de 15 dígitos. No entanto, por questões de aproximação numérica e truncamento durante os cálculos matriciais, os valores estritamente confiáveis estão na ordem de 12 dígitos. Recomenda-se utilizar este valor para análises físicas rigorosas.

5. Chamando os Métodos

Com o objeto `p` instanciado, os diversos métodos disponíveis podem ser utilizados acessando-os por meio da notação de ponto. A sequência padrão de execução é verificar os parâmetros, resolver o problema matematicamente e, em seguida, extrair as informações desejadas:

```
# Exibe no console os parâmetros configurados e o estado atual do espectro
p.is_described()

# Resolve o problema de autovalores e autovetores
# Este passo é obrigatório antes de gerar gráficos ou buscar resultados
p.is_solved()

# Recupera os dados (Exemplo: todos os autovalores de energia calculados)
autovalores = p.its_eigenvalues()
print(autovalores)
```

Após a execução bem-sucedida do método `is_solved()`, a biblioteca armazenará o espectro de energia internamente. A partir desse momento, métodos avançados para plotar gráficos de densidade, verificar a incerteza de Heisenberg e avaliar a ortogonalidade das funções já estarão liberados para uso.

MANUAL DE REFERÊNCIAS BIBLIOTECA PYSPECTRAL

A seguir, são apresentados os métodos que podem ser utilizados e exemplos ilustrativos para as suas chamadas.

SPECTRALMETHOD (CONSTRUTOR)

Inicializa um problema espectral, preparando o cálculo dos níveis de energia a partir das funções de potencial e peso fornecidas. Cria uma instância da classe, para a utilização dos métodos disponíveis.

Assinatura

```
def __init__(
    self,
    num_levels: int,
    length: float,
    f_function: Callable[[float], float],
    g_function: Callable[[float], float],
    num_digits: int = 15,
    weight: Optional[Callable[[float], float]] = None,
    root_filename: str = "output",
    label: str = "project",
    optimizer_L: str = 'n'
) -> None:
```

Parâmetros

- `num_levels (int)` – número de níveis (autofunções) a serem calculados.
- `length (float)` – comprimento do intervalo (L); deve ser positivo.
- `f_function (callable)` – função ($f(x)$) que define o potencial.
- `g_function (callable)` – função ($g(x)$) associada ao problema.
- `num_digits (int, opcional)` – precisão numérica dos cálculos; padrão é 15.
- `weight (callable, opcional)` – função peso ($w(x)$); se `None`, assume-se ($w(x) = 1$).
- `root_filename (str, opcional)` – prefixo para arquivos de saída, quando gerados; padrão “*output*”.
- `label (str, opcional)` – identificador do projeto, usado em títulos de arquivo; padrão “*project*”.
- `optimizer_L (str, opcional)` – controla otimização de `length`:
 - `'y'` : Otimiza (L) e o utiliza durante a execução. Caso contrário, utiliza `length` definido pelo usuário.

Retorno

- None. O construtor inicializa atributos internos e prepara o objeto para cálculos posteriores.

Exemplo de Uso

```
# importando o módulo spectral
from PySpectral import SpectralMethod

# Definindo um problema
# definição da função potencial
def f(x):
    return x**2

# definição da função g(x)
def g(x):
    return 1

# definição da função peso w(x)
def w(x):
    return 1

# inicia o objeto 'p'
p = SpectralMethod(
    num_levels=100,          # número de níveis de energia
    length=10.0,             # comprimento inicial
    f_function=f,            # função f (potencial)
    g_function=g,            # função g
    weight=w,                # função peso
    num_digits=15,           # dígitos considerados
    label="HalfOscillator",  # nome para o problema avaliado
    optimizer_L='y'          # seletor para otimizar o comprimento
)
```

Importante

Ao definir as funções que serão chamadas pelo método construtor é importante que seu rótulo/nome não seja igual a possíveis constantes declaradas a priori. Isso evita possíveis erros associados a ambiguidade no momento de chamada e criação da instância.

Outro detalhe importante diz respeito aos valores *default* das funções f , g e w . Caso elas não sejam referenciadas na instanciação do problema, por padrão, as funções assumirão o comportamento de uma função constante com retorno: $f = 0$, $g = 1$ e $w = 1$.

Por fim, vale destacar que este método utiliza o cálculo de autovalores e autovetores para a solução aproximada. Aumentar os níveis fará com que o esforço computacional aumente consideravelmente, já que as buscas ficam em torno de matrizes.

Observações

- A função `weight`, se não fornecida, é internamente definida como sendo a função constante igual a 1.

- `root_filename` e `label` aparecem em logs e nomes de arquivo; padronize-os para facilitar o rastreamento dos dados exportados ao longo do projeto.
- Otimizar `length(optimizer_L = 's')` pode aumentar a precisão dos autovalores, mas levar mais tempo de execução.
- Exceções
 - `ValueError`: se `num_levels` ou `length` for menor ou igual a zero.
 - `TypeError`: se `f_function` ou `g_function` não forem funções.

IS_DESCRIBED

Exibe no console os parâmetros configurados e o estado atual do espectro (calculado ou não).

Assinatura

```
def is_described(self) -> None
```

Parâmetros

- Nenhum parâmetro é exigido. Acessa a instância de `SpectralMethod` cujos atributos serão descritos.

Retorno

- Nenhum (`None`). A informação é impressa diretamente na saída padrão.

Exemplo de Uso

```
# chamada da função, considerando que uma instância SpectralMethod já tenha
# sido inicializada previamente.
p.is_described()

# exemplo de saída
Number of energy levels: 100
Interval amplitude: 6.592786280606971
f function: x**2
g function: 1
w (weight) function: 1
Label: HalfOscillator
Number of digits to be used: 15
The energy spectrum has not been calculated yet.
```

Observações:

- Se o espectro já foi calculado, via `calculate_spectrum`, a mensagem final indicará que o cálculo foi concluído e exibirá o intervalo otimizado, se aplicável.
- Útil para depuração e verificação rápida de parâmetros antes de rodar simulações longas.

IS_SOLVED

Resolve o problema de autovalores e autovetores baseado nos parâmetros configurados, calculando e armazenando internamente o espectro de energia e os vetores próprios.

Assinatura

```
def is_solved(self, number_of_digits: Optional[int] = None) -> None
```

Parâmetros

- `number_of_digits` (int, opcional) – precisão em dígitos para o cálculo.
 - Se fornecido e for um inteiro positivo, atualiza `self.num_digits` antes de iniciar o cálculo.
 - Se permanecer o valor default `None`, utiliza o valor previamente definido em `self.num_digits`.

Retorno

- `None`. (O método imprime mensagens de progresso e, ao término, atualiza atributos internos da classe, como `self.eigenvalues` e `self.eigenvectors`.)

Exemplos de Uso

```
from PySpectral import SpectralMethod

# supondo p já inicializado via SpectralMethod(...)

# usa self.num_digits definido no init da classe
p.is_solved()

# redefine precisão para 20 dígitos e resolve
p.is_solved(number_of_digits=20)

# saída esperada
Calculating eigenvalues and eigenvectors ...

Calculating energy eigenvalues.
Using 15 digits to begin with.

All eigenvalues are real. I've used 15 digits.
Time elapsed: 30.356478 seconds
Calculating eigenvectors ...

Calculating energy eigenvectors now ...
Time elapsed: 30.253860 seconds.
=====
The eigenvalue/eigenvector problem has been completely solved with 15 digits
used.
```

Observações

- Os tempos de execução são exibidos para diagnóstico de desempenho.
- Após chamada bem-sucedida, pode-se recuperar valores e vetores via métodos específicos (`its_eigenvalues`, `its_eigenvectors`).
- Chamadas subsequentes sem alterar parâmetros reaproveitam o espectro já calculado, a não ser que o parâmetro `number_of_digits` seja alterado.
- Após executar o método `is_solved()`, métodos como `check_equations()`, `wave_function()` e `probability_density()` já podem ser utilizados para análise adicional.
- Exceção:
 - `ValueError`: se `number_of_digits` for fornecido, mas não for um inteiro positivo.
 - `RuntimeError`: se ocorrer erro numérico durante a fatoração ou diagonalização das matrizes.

ITS_EIGENVALUES

Retorna os autovalores calculados, total ou parcialmente, facilitando acesso por índice, fatia ou intervalo. Imprime o resultado na tela.

Assinatura

```
def its_eigenvalues(self, *indices: Union[int, slice, range]) -> Union[List[float], float]
```

Parâmetros

- `*indices` – uma sequência de inteiros, fatia (`slice`) ou sequência (`range`) (base-1) indicando quais níveis retornarem.
 - Se não for passado nenhum índice, retorna todos os autovalores.

Retorno

- `float` : se retornando um único índice.
- `List[float]` : se retornando múltiplos ou todos os índices.

Exemplo de Uso

```
# depois de executado o comando p.is_solved()

# retorna todos os autovalores
all_vals = p.its_eigenvalues()

# retorna o primeiro autovalor
first = p.its_eigenvalues(1)

# retorna os três primeiros autovalores
slice_vals = p.its_eigenvalues(slice(1,4))

# retorna os autovalores selecionados
custom = p.its_eigenvalues(1, 3, 5)

# exemplo de saída esperada para o último caso
[3.00000000000001297, 11.000000000055795, 19.000000465040053]
```

Observações

- Internamente, os índices são convertidos de base-1 (conceitual) para base-0 (lista Python).
- Para grandes espectros, fatias muito amplas podem sobrecarregar a memória.
- Exceções:
 - `RuntimeError`: se chamado antes do método `is_solved()`.
 - `IndexError`: se qualquer índice estiver fora do intervalo $1 \leq i \leq \text{num_levels}$.

ITS_EIGENVECTORS

Retorna os autovetores já calculados. Se ainda não estiverem disponíveis, dispara internamente o método `calculate_eigenvectors`. Ao receber um único inteiro positivo, devolve apenas o autovetor correspondente (indexação base-1). Permite acesso por índice em base-1, fatias (`slices`), sequências (`ranges`) ou listas de índices. Sem argumentos, retorna a lista completa de autovetores.

Assinatura

`its_eigenvectors(*args) → list or array`

Parâmetros

- `*args` (*opcional*)
 - `int`: retorna o autovetor naquele índice (1 → primeiro, 2 → segundo etc.).
 - `slice` ou `range`: retorna lista de autovetores contíguos (ex.: `slice(1,4)` → índices 1,2,3).
 - `list[int]`: retorna lista de autovetores nos índices especificados.
 - Sem argumentos: retorna lista completa de autovetores.

Retorno

- Se `*args` contiver um único inteiro, retorna um único autovetor (array). Caso contrário, retorna uma lista de autovetores (cada um como array).

Exemplos de Uso

```
# a) Todos os autovetores
all_vs = p.its_eigenvectors()

# b) Primeiro autovetor
first_v = p.its_eigenvectors(1)

# c) Autovetores 1, 2 e 3 via slice
v_slice = p.its_eigenvectors(slice(1, 4))

# d) Autovetores 1, 2 e 3 via range
v_range = p.its_eigenvectors(range(1, 4))

# e) Autovetores nos índices 1, 3 e 5
v_list = p.its_eigenvectors(1, 3, 5)
```

Observações

- Converte automaticamente índices base-1 para base-0, facilitando a correspondência com notação matemática.
- Permite recuperar múltiplos autovetores em uma única chamada, usando diferentes tipos de índices.
- Garante cálculo único de autovetores graças ao atributo `self.has_eigenvectors_been_calculated`, que indica se o processo já foi realizado.
- Exceções:

- `TypeError`: Se um dos argumentos não for `int`, `slice`, `range` ou lista de inteiros (`list[int]`).
- `IndexError`: Se qualquer índice estiver fora do intervalo válido (menor que 1 ou maior que total de autovetores).

ITS_EIGENPAIRS

Retorna pares de autovalor e autovetor (λ, v) já calculados (ou dispara `calculate_eigenvectors()` se necessário). Permite seleção por índice único, fatia (`slice`), intervalo (`range`) ou listas de índices, sempre em indexação base-1. Opcionalmente, salva todos os resultados em um arquivo `.txt`.

Assinatura

```
def its_eigenpairs(self, *args, save_to_file: bool = True) -> list or [float, array]
```

Parâmetros

- `*args` (opcional)
 - `int`: retorna o par $[\lambda_i, v_i]$ correspondente ao índice fornecido ($1 \rightarrow$ primeiro, $2 \rightarrow$ segundo etc.).
 - `slice`: retorna lista de pares contíguos (ex.: `slice(1, 4)` $\rightarrow$ pares 1, 2 e 3).
 - `range`: retorna lista de pares em intervalo (ex.: `range(1, 4)` $\rightarrow$ pares 1, 2, 3).
 - `list[int]`: retorna lista de pares nos índices especificados.
 - Se não for fornecido nenhum argumento, retorna todos os pares disponíveis.
- `Save_to_file` (`bool`, padrão=`True`): se `True`, salva os pares em `<root_filename>_Eigenpairs.txt`, no formato:

```
# (1) Eigenvalue, (2) Eigenvector (full)
       $\lambda$           [v1, v2, v3, ...]
```

Retorno

- `[\lambda, v]`: se fornecido um único índice.
- `list[[\lambda, v]]`: se múltiplos índices ou nenhum argumento forem fornecidos.

Exemplos de Uso

```
# considerando que o problema já foi instanciado e resolvido.

# a) Todos os pares [\lambda, v]
all_pairs = p.its_eigenpairs()

# b) Primeiro par
first_pair = p.its_eigenpairs(1)

# c) Pares 1, 2 e 3 via slice
slice_pairs = p.its_eigenpairs(slice(1,4))

# d) Pares 1, 2 e 3 via range
range_pairs = p.its_eigenpairs(range(1,4))

# e) Pares nos índices 1, 3 e 4
selected_pairs = p.its_eigenpairs(1, 3, 4)

print(first_pair)

# Exemplo de saída:
```

```
# [0.123456789012, array([0.70710678, 0.70710678, 0.])]
```

Observações

- A indexação é base-1, para manter consistência com a notação matemática.
- Se os autovetores ainda não tiverem sido calculados, o método executa `calculate_eigenvectors` automaticamente.
- O arquivo salvo contém todos os pares selecionados, com vetores serializados em formato legível.
- O atributo `self.eigenvectors` deve estar disponível após cálculo.
- Exceções:
 - `ValueError`: se o índice fornecido for menor ou igual a zero ou se algum argumento não for suportado (`int`, `slice`, `range` ou `list[int]`).
 - `IndexError`: se algum índice estiver fora do intervalo válido (menor que 1 ou maior que total de pares disponíveis).

ITS_EIGENFUNCTION

Retorna a função de onda normalizada do nível i avaliada em x .

A função pode ser avaliada em um ponto numérico ou retornada como expressão simbólica. Se o valor passado como argumento x for numérico, retorna um número (utiliza a biblioteca *NumPy*); caso contrário, trata x como símbolo, constrói e retorna a expressão (utiliza a biblioteca *SymPy*¹).

A autofunção é definida como:

$$\psi_{i(x)} = \sqrt{\frac{2}{L}} \cdot \sum_{\{m=1\}}^N a_m \sin\left(\frac{m \pi x}{L}\right)$$

onde:

- `L = self.length` é o comprimento do intervalo,
- `N` é o número de coeficientes do autovetor,
- `am` são os coeficientes do autovetor associado ao nível i .

Assinatura

```
def its_eigenfunction(self, x, i, tolerance: float = 1e-15) -> float |  
np.ndarray | sympy.Expr
```

Parâmetros

- `x(float, numpy.ndarray ou str)`:
 - Se numérico (`float` ou `numpy.ndarray`), retorna valores numéricos da função de onda.
 - Se `string` (ex.: `"x"`), retorna uma expressão simbólica em *SymPy*.
- `i(int)`: índice do nível desejado (base-1).
- `tolerance(float, opcional, padrão=1e-15 (10-15))`: Coeficientes $|a_m|$ menores que esse valor são ignorados na soma.

Retorno

- `float`: se `x` for escalar numérico.
- `numpy.ndarray`: se `x` for *array NumPy*.
- `sympy.Expr`: se `x` for *string* (expressão simbólica).

Exemplo de Uso

```
# considerando que o problema já foi instanciado e resolvido.
```

¹ SymPy é uma biblioteca Python para matemática simbólica. Seu objetivo é se tornar um sistema de álgebra computacional (CAS) completo, mantendo o código o mais simples possível para ser compreensível e facilmente extensível. SymPy é escrito inteiramente em Python [2].

```

# 1) Avaliação numérica no ponto x=0.5, nível 2
val = p.its_eigenfunction(0.5, 2, tolerance=1e-6)
print(val)
# → 1.2345 (exemplo numérico)

# 2) Avaliação em vários pontos
xs = np.linspace(0, 1, 100)
vals = p.its_eigenfunction(xs, 1)
print(vals.shape)
# → (100,)

# 3) Expressão simbólica em x para o nível 1
psi1 = p.its_eigenfunction("x", 1, tolerance=1e-6)
print(psi1)
# → sqrt(2/L) * (a1*sin(pi*x/L) + a2*sin(2*pi*x/L) + ...)

```

Observações

- A normalização global ($\sqrt{2/L}$) garante ortonormalidade no poço quântico de comprimento L .
- Se os autovetores ainda não tiverem sido calculados, é necessário chamar antes `calculate_eigenvectors()` ou `is_solved()`.
- O parâmetro `tolerance` ajuda a eliminar coeficientes numéricos muito pequenos, acelerando o cálculo e simplificando a expressão simbólica.
- Para arrays de entrada (`numpy.ndarray`), o retorno é um array com os valores correspondentes da função de onda.
- Exceções:
 - `TypeError`: se i não for inteiro ou `tolerance` não for numérico.
 - `ValueError`:
 - Se $i < 1$ ou maior que o número de autovetores disponíveis.
 - Se os autovetores não tiverem sido calculados.

CALCULATE_SPECTRUM

Calcula o espectro de autovalores do problema generalizado de autovalores

$$\mathbf{A} \cdot \mathbf{v} = \lambda \cdot \mathbf{B} \cdot \mathbf{v}$$

Onde as matrizes **A** e **B** são construídas internamente pelos métodos privados `__build_matrix_d()` e `__build_matrix_d_prime()`.

O método utiliza `scipy.linalg.eig` para obter os autovalores, filtra valores inválidos (NaN, Inf) e descarta partes imaginárias desprezíveis. O resultado é armazenado em `self.en_spectrum` e pode ser salvo em arquivo ``.txt``.

Assinatura

```
def calculate_spectrum(self, save_to_file: bool = True) -> list
```

Parâmetros

- `save_to_file` (bool, opcional, padrão=True):
 - Se True, salva o espectro em um arquivo de texto. Cada linha contém o índice do nível e o valor do autovalor correspondente. Formato:

```
# NomeDoArquivo_Spectrum.txt
# Program version : X.Y
# Date: ...
# (1) Index, (2) Eigenvalue
```

Retorno

- `list`: lista ordenada de autovalores reais encontrados.

Exemplo de Uso

```
# considerando que o problema já foi instanciado e resolvido.

# Calcula o espectro de uma instância p
spectrum = p.calculate_spectrum(save_to_file=True)

# Exemplo de saída no console
```

```
Calculating energy eigenvalues.
Using 15 digits to begin with.
All eigenvalues are real. I've used 15 digits.
```

```
print(spectrum) # → [0.1234, 1.5678, 2.9012] valores ordenados
```

Observações

- Apenas autovalores reais e finitos são mantidos.

- Se o número de autovalores encontrados for diferente de `num_levels`, uma mensagem de aviso será exibida.
- Se algum autovalor apresentar parte imaginária não desprezível, também será emitido um aviso.
- O espectro é sempre retornado em ordem crescente.
- O atributo `self.has_spectrum_been_calculated` é definido como `True` após execução bem-sucedida.
- Exceções:
 - o `RuntimeError`: se `scipy.linalg.eig` falhar internamente (ex.: matrizes singulares).
 - o `ValueError`: se não for possível obter o número esperado de autovalores reais.

CALCULATE_EIGENVECTORS

Calcula os autovetores do problema generalizado de autovalores:

$$\mathbf{A} \cdot \mathbf{v} = \lambda \cdot \mathbf{B} \cdot \mathbf{v}$$

Onde as matrizes **A** e **B** são construídas internamente pelos métodos privados `__build_matrix_d()` e `__build_matrix_d_prime()`.

O método utiliza `scipy.linalg.eig` para obter autovalores e autovetores, ordena os resultados de acordo com os autovalores e normaliza cada autovetor de forma que tenha norma unitária, e o primeiro elemento não nulo seja positivo.

Os resultados são armazenados em `self.eigenvectors` como uma lista de pares $[\lambda, v]$. O atributo `self.has_eigenvectors_been_calculated` é definido como `True` após execução bem-sucedida.

Assinatura

```
def calculate_eigenvectors(self, save_to_file: bool = True) -> list
```

Parâmetros

- `save_to_file` (bool, opcional, padrão=True):
 - Se `True`, salva os pares $[\lambda, v]$ em `<root_filename>_Eigenvectors.txt`, no formato:

```
(1) Eigenvalue  (2) Eigenvector (full)
      λ                [v1, v2, v3, ...]
```

Retorno

- `list`: lista de pares $[\lambda, v]$, onde:
 - λ é o autovalor (float),
 - v é o autovetor normalizado (array NumPy).

Exemplo de Uso

```
# considerando que o problema já foi instanciado e resolvido.

# Primeiro calcula o espectro
p.calculate_spectrum()

# Agora calcula os autovetores
eigenvectors = p.calculate_eigenvectors(save_to_file=True)

# Exemplo de saída no console:
# _____
```



```
# Calculating energy eigenvectors now ...
# Eigenvectors saved to: output_Eigenvectors.txt
# _____
# Done!
#=====
```

Observações

- Se o espectro ainda não tiver sido calculado (`self.has_spectrum_been_calculated=False`), o método chama internamente `calculate_spectrum()`.
- Pequenas partes imaginárias (até tolerância 10^{-12}) são descartadas, convertendo os valores para reais.
- A normalização garante consistência de sinal e ortonormalidade.
- O arquivo salvo contém todos os pares $[\lambda, v]$ em formato legível.
- Exceções:
 - `RuntimeError`: se `scipy.linalg.eig` falhar (ex.: matrizes singulares).
 - `ValueError`: se ocorrer inconsistência no número de autovalores/autovetores calculados.

AN_EIGENFUNCTION

Avalia a função de onda $\psi(x)$ a partir dos coeficientes da base seno. A função é construída como combinação linear das funções de base senoidais, com normalização global $\sqrt{2/L}$.

A expansão é dada por:

$$\psi(x) = \sqrt{\frac{2}{L}} \cdot \sum_{m=1}^N c_m \sin\left(\frac{m\pi x}{L}\right)$$

onde:

- L é o comprimento do domínio,
- N é o número de coeficientes da base,
- c_m são os coeficientes do autovetor associado

Assinatura

```
def an_eigenfunction(self, x, basis_coeffs) -> float | np.ndarray
```

Parâmetros

- x (float ou array_like): Posição (ou posições) no espaço onde a função de onda será avaliada.
 - Se escalar, retorna um valor escalar.
 - Se array, retorna um array com os valores correspondentes.
- basis_coeffs (array_like, formato (N_basis,)): Coeficientes da base (componentes do autovetor), obtidos em `calculate_eigenvectors`.
- L (float): Comprimento do domínio. Recuperado automaticamente de `self.length`.

Retorno

- float: se x for escalar.
- `numpy.ndarray`: se x for array.

Exemplo de Uso

```
# Recupera os coeficientes do primeiro autovetor
coeffs = p.eigenvectors[0][1]

# Avaliação em um ponto escalar
val = p.an_eigenfunction(0.5, coeffs)
print(val)
# → 0.8732 (exemplo numérico)

# Avaliação em vários pontos
xs = np.linspace(0, 1, 100)
vals = p.an_eigenfunction(xs, coeffs)
```

```
print(vals.shape)
# → (100,)
```

Observações

- A função é útil para reconstruir a autofunção diretamente a partir dos coeficientes de base, sem depender de chamadas simbólicas.
- A normalização global $\sqrt{2/L}$ garante ortonormalidade no intervalo $[0, L]$.
- Exceções
 - `AttributeError`: se não for encontrado (ou não estiver definido) `self.length` na instância.

WAVE_FUNCTION

Constrói a função de onda $\psi(x, t)$ como combinação linear das autofunções previamente calculadas, com evolução temporal. Os coeficientes fornecidos determinam a contribuição de cada autofunção na superposição. Coeficientes com módulo abaixo da tolerância (`tolerance`) são tratados como zero.

A normalização é feita em relação à norma ponderada:

$$\int |\psi(x, t)|^2 w(x) dx$$

onde $w(x)$ é a função peso definida no problema

Assinatura

```
def wave_function(
    self,
    x,
    t,
    coefficients,
    tolerance: float = 1e-15,
    normalize: bool = True,
    save_to_file: bool = True
) -> Union[complex, float, np.ndarray, sympy.Expr]
```

Parâmetros

- `x` (float, array_like ou `sympy.Symbol`): Ponto(s) no espaço onde $\psi(x, t)$ será avaliada.
- `t` (float, array_like ou `sympy.Symbol`): Instante(s) de tempo para a evolução temporal.
- `coefficients` (list ou array de complex): Coeficientes da combinação linear das autofunções. Deve ter comprimento igual a `self.num_levels`.
- `tolerance` (float, opcional, padrão=1e-15): Coeficientes com módulo menor que esse valor são ignorados.
- `normalize` (bool, opcional, padrão=True): Se True, normaliza $\psi(x, t)$ de acordo com a norma ponderada.
- `save_to_file` (bool, opcional, padrão=True): Se True, salva os valores de $\psi(x, t)$ em `<root_filename>_WaveFunction.txt`.

Retorno

- complex ou float: se x e t forem escalares.
- `numpy.ndarray`: se x e/ou t forem arrays.
- `sympy.Expr`: se x ou t forem simbólicos.

Exemplo de Uso

```
[...]
# considerando que o problema já foi instanciado e resolvido.

# Coeficientes da superposição
```

```

coeffs = [1/np.sqrt(2), 0, 1/np.sqrt(2)]

# 1) Avaliação numérica em malha de pontos
xs = np.linspace(0, 1, 200)
ts = 0.1
psi_vals = p.wave_function(xs, ts, coeffs)
print(psi_vals.shape)
# -> (200,)

# 2) Avaliação simbólica
x, t = sp.symbols('x t')
psi_expr = p.wave_function(x, t, coeffs)
print(psi_expr)
# -> sqrt(2/L)*(c1*sin(pi*x/L)*exp(i*E1*t) + c3*sin(3*pi*x/L)*exp(i*E3*t))

```

Observações

- Coeficientes eliminados pelo atributo *tolerance* não contribuem para o somatório, acelerando a avaliação.
- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).
- A normalização pode ser custosa para malhas densas, pois envolve integração numérica.
- O arquivo salvo contém colunas no formato: “ x t $\psi(x,t)$ “
- A evolução temporal assume fator de fase ($e^{-iE_n t/\hbar}$).
- Exceções:
 - `RuntimeError`: se os autovetores não tiverem sido calculados.
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`

PROBABILITY_DENSITY

Calcula a densidade de probabilidade $|\psi(x,t)|^2$ para uma posição x , instante t e um conjunto de coeficientes que definem a combinação linear das autofunções.

A densidade de probabilidade é definida como:

$$\rho(x,t) = |\psi(x,t)|^2$$

onde $\psi(x,t)$ é a função de onda construída via método `wave_function`.

Assinatura

```
def probability_density(  
    self,  
    x,  
    t,  
    coefficients,  
    simplify_expr: bool = False,  
    save_to_file: bool = True  
  
    ) -> Union[sympy.Expr, float, np.ndarray]
```

Parâmetros

- `x` (float, array_like ou `sympy.Symbol`): Posição(ões) no espaço onde a densidade será avaliada.
- `t` (float, array_like ou `sympy.Symbol`): Instante(s) de tempo.
- `coefficients` (list ou array): Coeficientes da combinação linear das autofunções. Deve ter tamanho igual a `self.num_levels`.
- `simplify_expr` (bool, opcional, padrão=False): Se True, aplica `sympy.simplify` à expressão simbólica resultante.
- `save_to_file` (bool, opcional, padrão=True): Se True, salva os resultados em `<root_filename>_ProbabilityDensity.txt`.

Retorno

- `sympy.Expr`: se x e/ou t forem simbólicos.
- `float`: se x e t forem escalares numéricos.
- `numpy.ndarray`: se x e/ou t forem arrays.

Exemplo de Uso

```
from PySpectral import SpectralMethod  
import numpy as np  
import sympy as sp  
  
# Definição das funções f(x) e g(x)  
def f(x): return x**2
```

```

def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0, 0]

# 1) Avaliação simbólica
x, t = sp.symbols('x t')
rho_expr = p.probability_density(x, t, coeffs, simplify_expr=True)
print(rho_expr)
# -> expressão simbólica simplificada de  $|\psi(x,t)|^2$ 

# 2) Avaliação numérica em malha de pontos
xs = np.linspace(0, 1, 100)
ts = 0.2
rho_vals = p.probability_density(xs, ts, coeffs)
print(rho_vals.shape)
# -> (100,)

```

Observações

- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).
- O arquivo salvo contém colunas no formato: “x t $|\psi(x,t)|^2$ ”
- Se `x` e `t` forem arrays, o resultado será uma matriz 2D com dimensões `(len(t), len(x))`.
- Se apenas um dos parâmetros for array, o resultado será 1D.
- A densidade de probabilidade é sempre real e não-negativa.
- Exceções:
 - `RuntimeError`: Se as autofunções ou o espectro ainda não tiverem sido calculados.
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`

NORMALIZE_EIGENVECTOR

Normaliza um autovetor u de acordo com a definição do produto interno utilizado pela classe.

$$\left[\tilde{u} = \frac{u}{\sqrt{\langle u, u \rangle}}, \right]$$

onde $\langle u, u \rangle$ é calculado pelo método interno da biblioteca `__scalar_product`, que leva em consideração a função peso no cálculo do produto interno.

A função garante que o vetor resultante tenha norma unitária, a menos que a norma seja nula (ou quase nula), caso em que o vetor original é retornado sem alterações.

Assinatura

```
def normalize_eigenvector(self, u) -> np.ndarray
```

Parâmetros

- `u (list ou array_like)`: Vetor contendo os componentes do autovetor a ser normalizado. Deve ser unidimensional.

Retorno

- `numpy.ndarray`: vetor normalizado (mesma dimensão de entrada). Se a norma for zero (ou muito próxima de zero), retorna o vetor original sem normalização.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Recupera um autovetor
eigvec = p.eigenvectors[0][1]

# Normaliza manualmente
normalized_vec = p.normalize_eigenvector(eigvec)

print("Norma antes:", np.sqrt(p._SpectralMethod__scalar_product(eigvec,
eigvec)))
print("Norma depois:", np.sqrt(p._SpectralMethod__scalar_product(normalized_vec, normalized_vec)))
# → Norma antes: 2.345...
# → Norma depois: 1.0
```


Observações

- A função utiliza o produto interno definido em `__scalar_product`, garantindo consistência com a métrica usada no problema espectral.
- Se o vetor de entrada não for unidimensional, é lançada uma exceção.
- Útil para garantir consistência de escala antes de salvar ou manipular autovetores.
- Exceções:
 - `ValueError`: se o vetor fornecido não for unidimensional.
 - `TypeError`: Se `u` não for uma sequência ou *array* numérico adequado à representação de um vetor.

NORM_OF_WAVE_FUNCTION

Retorna uma função auxiliar que calcula a norma da função de onda $\psi(x, t)$, definida como a integral:

$$|\psi(t)|^2 = \int_0^L |\psi(x, t)|^2 w(x) dx$$

onde $w(x)$ é a função peso definida no problema. A norma é avaliada numericamente para cada instante de tempo t .

Assinatura

```
def norm_of_wave_function(self, coefficients) -> Callable[[float], float]
```

Parâmetros

- `coefficients` (list ou `numpy.ndarray`): Lista de coeficientes da combinação linear das autofunções (mesmo formato usado no método `wave_function`). Deve ter comprimento igual a `self.num_levels`.

Retorno

- `norm_func(t)` (Callable): Uma função que recebe um valor de tempo t e retorna o valor numérico da integral da norma da função de onda nesse instante.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função norma(t)
norm_func = p.norm_of_wave_function(coeffs)

# Avalia a norma em diferentes instantes
print(norm_func(0.0))    # → ~1.0 (normalizada)
print(norm_func(0.5))    # → ~1.0
print(norm_func(1.0))    # → ~1.0
```

Observações

- Garante a conservação de probabilidade ao longo da evolução temporal.
- Em sistemas com autovetores ortonormais no peso $w(x)$, espera-se $|\psi(t)|^2 = 1$ para qualquer t .
- A integração é realizada numericamente via quadratura de Gauss com alta precisão.
- A função retornada pode ser usada para verificar a conservação da norma ao longo do tempo.
- Se os coeficientes não forem válidos ou não corresponderem a `self.num_levels`, uma exceção será lançada.
- Exceções:
 - `ValueError`:
 - Se `coefficients` não for um array.
 - Se o número de coeficientes não corresponder a `self.num_levels`
 - `TypeError`: Se `coefficients` não for uma sequência de valores numéricos.

PLOT_EIGENFUNCTIONS

Plota as autofunções estacionárias $\psi_n(x)$ associadas a determinados níveis de energia e, opcionalmente, salva os dados correspondentes em arquivos de texto. Cada autofunção é reconstruída a partir dos coeficientes do autovetor correspondente e exibida em um gráfico

Assinatura

```
def plot_eigenfunctions(self,
    num_levels: int = None,
    levels: list[int] = None,
    x_points: int = 800,
    save: bool = True,
    save_txt: bool = True) -> None
```

Parâmetros

- `num_levels` (int, opcional): Número de níveis consecutivos a serem plotados, começando do primeiro nível. (Ex.: `num_levels=3` plota os níveis $n=0, 1, 2$. Ignorado se `levels` for fornecido).
- `levels` (list[int], opcional): Lista de índices (base-1) dos níveis específicos a serem plotados. (Ex.: `levels=[1, 3, 5]` plota os níveis $n=0, 2, 4$).
- `x_points` (int, padrão=800): Número de pontos no eixo x usados para avaliar cada autofunção.
- `save` (bool, padrão=True): Se True, salva a figura do gráfico no caminho `{root_filename}/{root_filename}_eigenfunctions.png`.
- `save_txt` (bool, padrão=True): Se True, salva os dados numéricos de cada autofunção (valores da posição x e $\psi(x)$) em arquivos de texto separados no formato `{root_filename}/{root_filename}_psi_level_{n}.txt`.

Retorno

- Nenhum (None). O método gera o gráfico das autofunções selecionadas e/ou salva os arquivos no diretório especificado.

Exemplo de Uso

```
from PySpectral import SpectralMethod

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=5, length=1.0, f_function=f, g_function=g)
p.is_solved()
```

```
# 1) Plotar os 3 primeiros níveis
p.plot_eigenfunctions(num_levels=3)

# 2) Plotar níveis específicos (1°, 3° e 5°), salvando a imagem, mas sem
arquivos .txt
p.plot_eigenfunctions(levels=[1, 3, 5], x_points=1000, save=True,
save_txt=False)
```

Observações

- É necessário que os autovetores já tenham sido calculados previamente no objeto instanciado.
- O domínio de plotagem no eixo x é definido automaticamente a partir do atributo `self.length`.
- O título do gráfico indica os níveis plotados em notação $n = 0, 1, 2, \dots$ ($n=0$, indica o estado fundamental).
- O parâmetro `save=True` exporta a figura em alta resolução (`dpi=900`) e com margens ajustadas (`bbox_inches="tight"`).
- Os arquivos texto gerados quando `save_txt=True` contêm um cabeçalho informativo com o nível da autofunção, o autovalor de energia e o tamanho do domínio L .
- Exceções
 - `ValueError`:
 - Se `num_levels < 1`.
 - Se nenhum dos parâmetros `num_levels` ou `levels` for fornecido.
 - `AttributeError`: se o comprimento do domínio (`self.length`) não estiver definido na instância antes da plotagem.

PLOT_EIGENFUNCTION_3D

Plota a autofunção $\psi_n(x)$ como uma superfície 3D para um único nível de energia. O eixo y é apenas auxiliar (sem significado físico), usado para dar largura à superfície e facilitar a visualização.

Assinatura

```
def plot_eigenfunction_3d(  
    self,  
    level: int,  
    x_points: int = 800,  
    y_points: int = 50,  
    show: bool = True,  
    save: bool = True  
) -> None
```

Parâmetros

- `x_points` (int, padrão=800): Número de pontos no eixo x usados para avaliar $\psi_n(x)$.
- `y_points` (int, padrão=50): Número de pontos no eixo auxiliar y, usado apenas para dar espessura à superfície.
- `show` (bool, padrão=True): Se True, exibe o gráfico na tela.
- `save` (bool, padrão=True): Se True, salva a figura em `<root_filename>_wavefunction3D_level{level}.png`.

Retorno

- None. O método gera e exibe (ou salva) a superfície 3D da autofunção.

Exemplo de Uso

```
from PySpectral import SpectralMethod  
  
# Definição das funções f(x) e g(x)  
def f(x): return x**2  
def g(x): return 1  
  
# Instancia e resolve o problema  
p = SpectralMethod(num_levels=5, length=1.0, f_function=f, g_function=g)  
p.is_solved()  
  
# Plota a autofunção do 2º nível de energia  
p.plot_eigenfunction_3d(level=2, x_points=500, y_points=100, show=True,  
    save=False)
```

Observações

- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).

- O domínio $x \in [0, L]$ é definido automaticamente a partir de `self.length`.
- O eixo y não possui interpretação física, servindo apenas para dar profundidade ao gráfico.
- O título do gráfico indica o nível de energia em notação $n = \text{level} - 1$.
- O parâmetro `save` salva a figura em alta resolução (`dpi=300`).
- O parâmetro `show=False` é útil para execução em lote (sem abrir janelas gráficas)
- Exceções
 - `AttributeError`: se `self.length` não estiver definido

PROBABILITY_DENSITY_IS_PLOTTED

Plota a densidade de probabilidade $|\psi(x, t)|^2$ em função da posição x .

Pode operar em dois modos distintos:

1. **Estático:** quando t é um instante único (valor real), gera um gráfico fixo da densidade naquele exato momento.
2. **Animação:** quando t é um intervalo (tupla ou lista), gera uma animação da evolução temporal da densidade de probabilidade, com a opção de exibir na tela ou salvar como GIF.

Assinatura

```
def probability_density_is_plotted(
    self,
    t,
    coefficients,
    num_frames: int = None,
    num_slices: int = None,
    save: bool = False,
    **plotopts
) -> None
```

Parâmetros

- t (float ou tuple/list[float]):
 - Valor único (ex: 0.5): plota a densidade $|\psi(x, t)|^2$ em um instante fixo no tempo.
 - Intervalo (ex: (0, 2)): gera animação da evolução temporal entre o tempo inicial e o tempo final.
- `coefficients` (list[complex]): Lista de coeficientes complexos da superposição de autofunções. O tamanho da lista deve ser obrigatoriamente igual a `self.num_levels`.
- `num_frames` (int, opcional): Número de quadros (frames) da animação. O padrão é 100 (utilizado apenas no modo animação).
- `num_slices` (int, opcional): Número de pontos (fatias) no eixo x para avaliar a função. O padrão é 200 para ambos os modos.
- `save` (bool, padrão=False): Se True, salva a animação como um arquivo GIF (a 15 fps) (ou o gráfico estático como uma imagem PNG (a 150 dpi) no diretório do projeto. Se False, exibe o resultado diretamente na tela.
- `**plotopts` (dict, opcional): Opções adicionais de plotagem, como "color", "title", "xlabel", "ylabel", "figsize".

Retorno

- None. O método exibe ou salva o gráfico/animação.

Exemplo de Uso


```

from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0, 0]

# 1) Gráfico estático em t=0.5 (exibido na tela)
p.probability_density_is_plotted(0.5, coeffs, color="blue",
title="Densidade em t=0.5")

# 2) Animação entre t=0 e t=2, com 50 quadros (salva como GIF)
p.probability_density_is_plotted((0, 2), coeffs, num_frames=50, save=True)

```

Observações

- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).
- O eixo x é vetorizado e definido automaticamente no intervalo de 0 até `self.length`.
- O título e os rótulos dos eixos do gráfico/animação podem ser personalizados através dos argumentos de `plotopts` (ex: `xlabel="Posição", figsize=(10, 8)`).
- **Otimização da Animação:** No modo animação, todos os quadros são pré-calculados antes da plotagem para garantir maior fluidez.
- **Coerência Física:** Para animações, os limites do eixo y são ajustados globalmente para todos os quadros, adotando uma margem de 10% acima do pico máximo de densidade de probabilidade encontrado em todo o intervalo de tempo (`global_max * 1.1`).
- No modo estático, a base do eixo y é sempre fixada em zero (`bottom=0`), pois a densidade de probabilidade nunca é negativa.
- Os arquivos salvos (quando `save=True`) são nomeados automaticamente com o prefixo `root_filename` da instância (ex: `meu_projeto_prob_density.gif`).
- Exceções
 - `ValueError`: se o número de coeficientes fornecidos na lista não for exatamente igual ao número de níveis (`self.num_levels`) configurado na instância.
 - `AttributeError`: se o atributo `self.length` não estiver definido antes da chamada do método.

PROBABILITY_DENSITY_3D

Plota a densidade de probabilidade $|\psi(x, t)|^2$ num gráfico 3D, mostrando a sua evolução no espaço e no tempo de forma contínua.

Eixos:

- **Eixo x:** posição (0 até `self.length`)
- **Eixo y:** tempo (do início ao fim do intervalo fornecido em `t_interval`)
- **Eixo z:** densidade de probabilidade $|\Psi(x, t)|^2$

Assinatura

```
def probability_density_3d(
    self,
    t_interval: tuple[float, float],
    coefficients: list[complex],
    num_frames: int = 50,
    num_slices: int = 200,
    animate_rotation: bool = False,
    save_as_gif: bool = True,
    gif_filename: str = None,
    **plotopts
) -> tuple
```

Parâmetros

- `t_interval` (tuple(float, float)): Intervalo de tempo (t_1, t_2) para o qual a evolução espacial e temporal será calculada.
- `coefficients` (list[complex]): Lista de coeficientes complexos da função de onda (superposição). O tamanho da lista deve ser obrigatoriamente igual a `self.num_levels`.
- `num_frames` (int, padrão=50): Número de pontos no eixo do tempo (resolução no eixo temporal).
- `num_slices` (int, padrão=200): Número de pontos no eixo da posição (resolução no eixo espacial).
- `animate_rotation` (bool, padrão=False): Se True, gera uma animação rotacionando a superfície 3D.
- `save_as_gif` (bool, padrão=True): Se True, salva a animação como um arquivo GIF.
- `gif_filename` (str, opcional): Caminho e nome do arquivo GIF. Se não fornecido, usa o padrão: `{root_filename}/{root_filename}_probability_density_3d.gif`.
- `**plotopts` (dict, opcional): Opções extras de plotagem e formatação da figura, como "cmap", "figsize", "xlabel", "ylabel", "zlabel", "title".

Retorno

- Retorna uma tupla contendo 5 elementos:
 - `fig` (matplotlib.figure.Figure): objeto da figura gerada.

- `ax(matplotlib.axes._subplots.Axes3DSubplot)`: objeto do eixo 3D.
- `density(numpy.ndarray)`: matriz 2D contendo os valores calculados da densidade.
- `t_values(numpy.ndarray)`: array com os valores de tempo utilizados no eixo y.
- `x_values(numpy.ndarray)`: array com os valores de posição utilizados no eixo x.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Plota densidade de probabilidade em 3D no intervalo t ∈ [0, 2]
fig, ax, density, t_vals, x_vals = p.probability_density_3d(
    t_interval=(0, 2),
    coefficients=coeffs,
    num_frames=40,
    num_slices=150,
    animate_rotation=True,
    save_as_gif=True,
    gif_filename="rho3d.gif",
    cmap="plasma",
    title="Densidade de Probabilidade em 3D")
```

Observações

- É necessário que os autovetores já tenham sido calculados previamente na instância (`calculate_eigenvectors()` ou `is_solved()`).
- O cálculo da superfície 3D é otimizado. A matriz de densidade é pré-alocada e preenchida iterando sobre o tempo, mas aplicando vetorização no espaço para acelerar o processo.
- A barra de cores (*colorbar*) é adicionada automaticamente à figura para facilitar a leitura dos valores da densidade no eixo Z.
- Se `animate_rotation=True` ou `save_as_gif=True`, a câmara realiza uma rotação automática de 360 graus, criando um efeito visual de perspectiva da superfície.
- Se `save_as_gif=True`, a animação é guardada (a 15 fps) e a figura é fechada na memória (`plt.close(fig)`) para libertar recursos de sistema. Caso contrário, o gráfico é exibido na tela (`plt.show()`).

- O mapa de cores (*colormap*) padrão é "viridis", mas pode ser facilmente alterado através dos argumentos do `plotopts` (ex: `cmap="plasma"`, `cmap="inferno"`).
- Exceções
 - `ValueError`: se o número de coeficientes fornecidos na lista não for exatamente igual a `self.num_levels`.
 - `AttributeError`: se o atributo de comprimento do domínio (`self.length`) não estiver definido na instância.

PLOT_WAVEFUNCTION_AND_DENSITY

Plota, lado a lado, o módulo da função de onda $|\Psi(x, t)|$ e a densidade de probabilidade $|\Psi(x, t)|^2$ num determinado domínio x .

Pode operar em dois modos:

1. **Estático:** quando t é um instante único, gera dois gráficos fixos.
2. **Animação:** quando t é um intervalo (t_1, t_2) , gera uma animação mostrando a evolução temporal, com opção de salvar como GIF.

Assinatura

```
def plot_wavefunction_and_density(  
    self,  
    t,  
    coefficients,  
    num_frames: int = None,  
    num_slices: int = None,  
    save_as_gif: bool = False,  
    gif_filename = None,  
    **plotopts  
) -> None
```

Parâmetros

- t (float ou tuple[float, float]):
 - Valor único: plota o módulo $|\Psi(x, t)|$ e a densidade de probabilidade $|\Psi(x, t)|^2$ em um instante fixo.
 - Intervalo (t_1, t_2) : gera animação da evolução temporal.
- `coefficients` (list[complex]): Lista de coeficientes da superposição de autofunções. Deve ter tamanho igual a `self.num_levels`.
- `num_frames` (int, opcional): Número de quadros da animação (padrão: 100).
- `num_slices` (int, opcional): Número de pontos no eixo x (padrão: 200).
- `save_as_gif` (bool, opcional, padrão=False): Se True, salva a animação como GIF (ou o gráfico estático como imagem).
- `gif_filename` (str, opcional, padrão="wavefunction_density.gif"): Nome do arquivo de saída.
- `**plotopts` (dict, opcional): Opções adicionais de plotagem, como "title", "xlabel", "ylabel", "figsize".

Retorno

- Nenhum (None) . O método exibe ou salva os gráficos/animação no diretório.

Exemplos de Uso

```
from PySpectral import SpectralMethod  
import numpy as np
```

```

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# 1) Gráfico estático em t=0.5
p.plot_wavefunction_and_density(0.5, coeffs, figsize=(12,5))

# 2) Animação entre t=0 e t=2, com 50 quadros (salva como GIF)
p.plot_wavefunction_and_density((0, 2), coeffs, num_frames=50,
save_as_gif=True, gif_filename="psi_density.gif")

```

Observações

- É necessário que os autovetores já tenham sido calculados previamente na instância (`calculate_eigenvectors()` ou `is_solved()`).
- O eixo x é vetorizado e definido automaticamente no intervalo de 0 até `self.length`.
- O gráfico mostra lado a lado:
 - Esquerda: módulo da função de onda $|\Psi(x, t)|$.
 - Direita: densidade de probabilidade $|\Psi(x, t)|^2$.
- Para manter a coerência física e evitar que os gráficos "saltem" durante a animação, os limites do eixo vertical (y) são fixos globalmente com uma margem de segurança de 10% acima do pico máximo encontrado em todo o intervalo de tempo estudado.
- O parâmetro `save_as_gif=True` salva a saída em arquivo (GIF para animações, PNG para gráficos estáticos) no diretório especificado.
- Exceções
 - `ValueError`: se o número de coeficientes não corresponder exatamente a `self.num_levels`.
 - `AttributeError`: se `self.length` não estiver definido antes da chamada ao método.

PROBABILITY_DENSITY_CARTOON_IS_SHOWN

Gera uma visualização estilo “cartoon” da densidade de probabilidade $|\psi(x, t)|^2$. A densidade é representada como uma imagem 2D, onde regiões de maior probabilidade aparecem mais escuras (valores negativos são usados para realçar essas regiões).

Pode operar em dois modos:

1. **Estático:** quando t é um instante único, gera uma imagem fixa.
2. **Animação:** quando t é um intervalo $[t_1, t_2]$, gera uma animação mostrando a evolução temporal, com opção de salvar como GIF.

Assinatura

```
def probability_density_cartoon(  
    self,  
    t,  
    coefficients,  
    num_slices: int = None,  
    save_as_gif: bool = True,  
    gif_fps: int = 30,  
    cmap: str = "plasma",  
    **plotopts  
) -> None
```

Parâmetros

- t (float ou tuple[float, float]):
 - Valor único: plota a densidade em um instante fixo.
 - Intervalo $[t_1, t_2]$: gera animação da evolução temporal.
- `coefficients` (list[complex]): Lista de coeficientes da superposição de autofunções. Deve ter tamanho igual a `self.num_levels`.
- `num_slices` (int, opcional): Número de amostras no tempo para a animação (padrão: 10).
- `save_as_gif` (bool, opcional, padrão=True): Se True, salva a saída como GIF (tanto no modo estático quanto animado).
- `gif_fps` (int, opcional, padrão=30): Taxa de quadros por segundo do GIF.
- `cmap` (str, opcional, padrão="plasma"): Colormap usado no `imshow` (ex.: "viridis", "inferno", "plasma").
- `**plotopts` (dict, opcional): Opções adicionais passadas ao `plt.imshow` (ex.: `vmin`, `vmax` etc.).

Retorno

- None. O método exibe ou salva a imagem/animação.

Exemplos de Uso

```

from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# 1) Visualização estática em t=0.5
p.probability_density_cartoon(0.5, coeffs, cmap="viridis",
save_as_gif=False)

# 2) Animação entre t=0 e t=2
p.probability_density_cartoon((0, 2), coeffs, num_slices=30,
save_as_gif=True, gif_fps=15)

```

Observações

- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).
- O eixo x é definido automaticamente no intervalo $[0, L]$.
- A densidade é replicada verticalmente para formar uma “faixa colorida” que facilita a visualização.
- O parâmetro `cmap` controla o esquema de cores.
- O parâmetro `save_as_gif=True` salva a saída em arquivo GIF (mesmo no modo estático).
- Exceções
 - `ValueError`: se t não for um número ou um intervalo $[t_1, t_2]$.
 - `AttributeError`: se `self.length` não estiver definido.
 - `RuntimeError`: Se os autovalores/autovetores não estiverem calculados (não foi executado `is_solved()`).

PLOT_WAVEFUNCTION_SNAPSHOT

Plota um "instantâneo" (*snapshot*) da função de onda num instante de tempo fixo t . O método exibe o gráfico na tela e permite, opcionalmente, guardar a imagem do gráfico e exportar os dados numéricos associados para um ficheiro de texto.

Assinatura

```
def plot_wavefunction_snapshot(self,
    t: float,
    coefficients,
    num_slices: int = 1000,
    save_data: bool = False,
    data_filename: str = None,
    plot_filename: str = None,
    **plotopts) -> None
```

Parâmetros

- `t` (float): Instante de tempo fixo para o qual a função de onda será avaliada e desenhada.
- `coefficients` (list[complex]): Lista de coeficientes complexos da superposição de autofunções.
- `num_slices` (int, padrão=1000): Número de pontos no eixo da posição (x) para avaliar a função, garantindo a suavidade da curva.
- `save_data` (bool, padrão=False): Se True, o método guarda a imagem do gráfico e exporta os dados espaciais e de amplitude para um ficheiro de texto .txt.
- `data_filename` (str, opcional): Nome ou caminho personalizado para o ficheiro de dados .txt. Se não for fornecido e `save_data=True`, será gerado um nome automático.
- `plot_filename` (str, opcional): Nome ou caminho personalizado para a imagem do gráfico (.png). Se não for fornecido e `save_data=True`, será gerado um nome automático.
- `**plotopts` (dict, opcional): Opções adicionais de formatação da figura (ex.: `figsize`).

Retorno

- Nenhum (None). O método exibe o gráfico e, se solicitado, guarda os ficheiros em disco.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
```

```

p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Plota um snapshot no tempo t=1.5, guardando imagem de alta resolução e os
dados TXT
p.plot_wavefunction_snapshot(
    t=1.5,
    coefficients=coeffs,
    num_slices=1500,
    save_data=True,
    figsize=(10, 6)
)

```

Observações

- Quando `save_data=True`, a imagem é exportada com uma resolução alta (`dpi=2000`), sendo ideal para publicações científicas, artigos ou teses.
- Se os nomes dos ficheiros (`data_filename` e `plot_filename`) não forem explicitamente fornecidos, o sistema gera nomes automáticos que incluem um *timestamp* (data e hora no formato `YYYYMMDD_HHMMSS`), evitando a substituição acidental de execuções anteriores.
- O ficheiro numérico gerado (`.txt`) contém um cabeçalho informativo seguido de três colunas de dados tabulados:
 1. Posição (x)
 2. Valor complexo da função de onda (Ψ)
 3. Densidade de probabilidade ($|\Psi|^2$)

Exceções

- **AttributeError:** Se o atributo de comprimento do domínio (`self.length`) não estiver definido previamente na instância.
- **Erro de I/O:** Se `save_data=True`, o método tentará escrever os ficheiros no disco. Caso ocorra um problema de permissão ou de diretório inexistente ao exportar o `.txt`, o erro será capturado pela rotina `try...except` e a mensagem de erro será mostrada, sem interromper a execução do programa.

PLOT_WAVEFUNCTION_3D

Plota a evolução temporal do módulo da função de onda $|\Psi(x, t)|$ num gráfico de superfície 3D. Este método permite visualizar de forma contínua o comportamento da função de onda ao longo do espaço e do tempo.

- Eixo X: Posição (do domínio 0 até `self.length`).
- Eixo Y: Tempo (t), variando dentro do intervalo fornecido.
- Eixo Z: Módulo da função de onda $|\Psi(x, t)|$.

Assinatura

```
def plot_wavefunction_3d(self,
    t_span: tuple,
    coefficients,
    num_x: int = 100,
    num_t: int = 100,
    save_data: bool = False,
    data_filename: str = None,
    plot_filename: str = None,
    elev=30,
    azim=-45,
    **plotopts) -> None
```

Parâmetros

- `t_span (tuple)`: Intervalo de tempo (`t_start`, `t_end`) que define o período da evolução temporal a ser simulado.
- `coefficients (list[complex])`: Lista de coeficientes complexos da expansão da função de onda (superposição).
- `num_x (int, padrão=100)`: Resolução espacial da grelha (número de pontos calculados no eixo da posição XX).
- `num_t (int, padrão=100)`: Resolução temporal da grelha (número de instantes calculados no eixo do tempo YY).
- `save_data (bool, padrão=False)`: Se `True`, o método guarda a figura do gráfico 3D gerado e exporta a matriz de dados para um ficheiro de texto (.txt).
- `data_filename (str, opcional)`: Caminho ou nome personalizado para o ficheiro de dados .txt. Se não for especificado, será gerado um nome com um carimbo de tempo (*timestamp*).
- `plot_filename (str, opcional)`: Caminho ou nome personalizado para a imagem .png. Se não for especificado, será gerado um nome automático com *timestamp*.
- `elev (int/float, padrão=30)`: Ângulo de elevação inicial da perspectiva da visualização 3D (em graus).

- `azim` (int/float, padrão=-45): Ângulo de azimute inicial da perspectiva da visualização 3D (em graus).
- `**plotopts` (dict, opcional): Opções extra de formatação do gráfico, como "figsize".

Retorno

- Nenhum (None). O método exibe o gráfico interativo no ecrã e, caso seja solicitado, guarda as imagens e os dados no disco.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções do sistema
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Plota a evolução da função de onda no intervalo  $t \in [0, 5]$ 
# Guarda os dados e o gráfico com uma resolução espacial/temporal de 150x150
p.plot_wavefunction_3d(
    t_span=(0, 5),
    coefficients=coeffs,
    num_x=150,
    num_t=150,
    save_data=True,
    elev=40,
    azim=-60
)
```

Observações

- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).
- O gráfico gerado inclui as projeções topográficas (*contour curves*) no "chão" (base do eixo Z) para facilitar a leitura da magnitude em diferentes posições e instantes.
- O visual do gráfico 3D é otimizado para publicações, utilizando painéis de fundo transparentes e uma paleta de cores (`colormap`) do tipo "viridis", que é acompanhada por uma barra de cores lateral devidamente legendada.

- Se `save_data=True`, a imagem é exportada com uma resolução alta (`dpi=2000`) utilizando margens ajustadas (`bbox_inches='tight'`).
- Os ficheiros numéricos exportados no formato `.txt` assumem a estrutura de uma matriz longa (lista de pontos), contendo colunas para: Tempo (t), Posição (x) e Módulo ($|\Psi|$).
- A nomenclatura automática dos ficheiros recorre à biblioteca `datetime` para evitar a sobreposição de resultados de execuções anteriores, gerando nomes no formato `_Evol3D_YYYYMMDD_HHMMSS.png`.

Exceções

- `AttributeError`: Se o atributo `self.length` (comprimento do domínio) não estiver definido na instância instanciada.
- Erro de I/O: Caso não seja possível escrever os ficheiros exportados na diretoria (ex: falta de permissões), a exceção é capturada internamente (`try...except`) e um aviso é impresso na consola sem interromper a execução do restante código.

PLOT_PROBABILITY_DENSITY_3D_STATIC

Gera uma figura 3D **estática** da evolução temporal da densidade de probabilidade $|\Psi(x, t)|^2$. Ao contrário de outras funções de visualização, este método foi desenhado com um foco rigoroso na alta qualidade visual (*clean design*), sendo ideal para exportação para publicações científicas, artigos e impressão.

- Eixo X: Posição (do domínio 0 até `self.length`).
- Eixo Y: Tempo (t), variando de acordo com o intervalo fornecido. Visualmente, este eixo é invertido na figura gerada.
- Eixo Z: Densidade de probabilidade $|\Psi(x, t)|^2$.

Assinatura

```
def plot_probability_density_3d_static(self,
    t_interval: tuple,
    coefficients,
    num_t: int = 100,
    num_x: int = 200,
    save_data: bool = False,
    data_filename: str = None,
    plot_filename: str = None,
    elev=30,
    azim=135,
    **plotopts) -> tuple
```

Parâmetros

- `t_interval` (tuple): Intervalo de tempo (t_1 , t_2) que define o período da evolução a ser avaliado.
- `coefficients` (list[complex]): Lista de coeficientes complexos da expansão da função de onda.
- `num_t` (int, padrão=100): Resolução temporal da grelha (número de instantes no tempo calculados).
- `num_x` (int, padrão=200): Resolução espacial da grelha (número de posições no espaço calculadas).
- `save_data` (bool, padrão=False): Se True, o método guarda a figura do gráfico e exporta a matriz de dados resultante para um ficheiro de texto (.txt).
- `data_filename` (str, opcional): Caminho ou nome personalizado para o ficheiro de dados .txt. Se não for especificado, será gerado um nome automático com *timestamp*.
- `plot_filename` (str, opcional): Caminho ou nome personalizado para a imagem .png. Se não for especificado, será gerado um nome automático com *timestamp*.
- `elev` (int/float, padrão=30): Ângulo de elevação inicial da perspectiva da visualização (em graus).

- `azim` (int/float, padrão=135): Ângulo de azimute inicial da perspectiva da visualização (em graus).
- `**plotopts` (dict, opcional): Opções adicionais de formatação, como "figsize", "cmap" (mapa de cores, por defeito "viridis"), "xlabel", "ylabel", "zlabel" e "title".

Retorno

Retorna uma tupla contendo 2 elementos do Matplotlib:

- `fig` (matplotlib.figure.Figure): Objeto principal da figura.
- `ax` (matplotlib.axes._subplots.Axes3DSubplot): Objeto do eixo 3D, permitindo manipulações posteriores pelo utilizador, se necessário.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Funções do problema
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0, 0]

# Plota e guarda uma superfície estática em alta qualidade (dpi=300)
# Perfeita para incluir num documento académico
fig, ax = p.plot_probability_density_3d_static(
    t_interval=(0, 3),
    coefficients=coeffs,
    num_t=150,
    num_x=250,
    save_data=True,
    cmap="plasma"
)
```

Observações

- É necessário que os autovetores já tenham sido calculados (`calculate_eigenvectors()` ou `is_solved()`).
- Quando `save_data=True`, a figura é guardada com a resolução configurada para `dpi=300` e possui as margens otimizadas (`bbox_inches='tight'`).
- Os dados numéricos exportados no ficheiro .txt assumem a estrutura de matriz longa, com colunas correspondentes a: Tempo (t), Posição (x) e Densidade de Probabilidade ($|\Psi|^2$).

- Os nomes dos ficheiros gerados automaticamente incluem um *timestamp* para garantir que dados de múltiplas execuções não se sobreponham acidentalmente.

Exceções

- `AttributeError`: Caso o atributo do comprimento do domínio (`self.length`) não tenha sido definido antes do plot.
- Erro de I/O: Se a opção `save_data=True` for passada e ocorrer um problema de acesso ou permissão na diretoria, o método irá capturar a exceção e apresentar um aviso na consola, sem travar a execução do código (o gráfico continuará a ser exibido na tela).

EXPECTED_POSITION

Calcula o valor esperado da posição $\langle x \rangle(t)$ para uma função de onda $\psi(x, t)$ construída a partir de uma superposição de autofunções com coeficientes fornecidos.

Por definição:

$$\langle x \rangle(t) = \frac{\int_0^L x w(x) |\psi(x, t)|^2 dx}{\int_0^L w(x) |\psi(x, t)|^2 dx}$$

onde:

- L é o comprimento do domínio,
- $w(x)$ é a função peso definida no problema,
- $|\psi(x, t)|^2$ é a densidade de probabilidade.

Assinatura

```
def expected_position(self, coefficients) -> Callable[[float], float]
```

Parâmetros

- `coefficients` (list ou `numpy.ndarray`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.

Retorno

- `exp_pos(t)` (Callable): uma função que recebe um instante de tempo `t` e retorna o valor esperado da posição $\langle x \rangle(t)$.

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função <x>(t)
exp_pos = p.expected_position(coeffs)

# Avalia em diferentes instantes
print(exp_pos(0.0))    # → valor esperado em t=0
print(exp_pos(0.5))    # → valor esperado em t=0.5
```

```
print(exp_pos(1.0))    # → valor esperado em t=1.0
```

Observações

- A integração é feita numericamente via quadratura de Gauss com alta precisão.
- O denominador garante a normalização da função de onda.
- Se o denominador for zero (função de onda nula ou mal definida), é lançada uma exceção.
- Útil para estudar a evolução temporal da posição média de uma partícula em um poço quântico.
- Exceções
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`.
 - `ZeroDivisionError`: se o denominador da normalização for zero.
 - `RuntimeError`: Se os autovalores/autovetores não estiverem disponíveis (Ex.: o método `is_solved()` não foi executado).

EXPECTED_POSITION_SQUARED

Calcula o valor esperado do quadrado da posição $\langle x^2 \rangle(t)$ para uma função de onda $\psi(x, t)$ construída a partir de uma superposição de autofunções com coeficientes fornecidos.

Por definição:

$$\langle x^2 \rangle(t) = \frac{\int_0^L x^2 w(x) |\psi(x, t)|^2 dx}{\int_0^L w(x) |\psi(x, t)|^2 dx}$$

onde:

- L é o comprimento do domínio,
- $w(x)$ é a função peso definida no problema,
- $|\psi(x, t)|^2$ é a densidade de probabilidade.

Assinatura

```
def expected_position_squared(self, coefficients) -> Callable[[float], float]
```

Parâmetros

- `coefficients` (list ou `numpy.ndarray`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.

Retorno

- `exp_x2(t)` (Callable): Uma função que recebe um instante de tempo t e retorna o valor esperado $\langle x^2 \rangle(t)$.

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função <x^2>(t)
exp_x2 = p.expected_position_squared(coeffs)

# Avalia em diferentes instantes
print(exp_x2(0.0))    # → valor esperado de x^2 em t=0
print(exp_x2(0.5))    # → valor esperado de x^2 em t=0.5
print(exp_x2(1.0))    # → valor esperado de x^2 em t=1.0]
```

Observações

- A integração é feita numericamente via quadratura de Gauss com alta precisão.
- O denominador garante a normalização da função de onda.
- Se o denominador for zero (função de onda nula ou mal definida), é lançada uma exceção.
- Essa função é útil para calcular a variância da posição: $\sigma_x^2 = \langle x^2 \rangle - \langle x \rangle^2$, em conjunto com o método `expected_position`.
- Exceções
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`.
 - `RuntimeError`: Se as autofunções ou o espectro ainda não tiverem sido calculados (Ex.: `is_solved()` não foi executado).

POSITION_UNCERTAINTY

Calcula a incerteza na posição $\sigma_x(t)$ para um pacote de onda $\psi(x, t)$ construído a partir de uma superposição de autofunções com coeficientes fornecidos.

Por definição:

$$\sigma_x(t) = \sqrt{\langle x^2 \rangle(t) - \langle x \rangle(t)^2}$$

onde:

- $\langle x \rangle(t)$ é o valor esperado da posição,
- $\langle x^2 \rangle(t)$ é o valor esperado do quadrado da posição.

Assinatura

```
def position_uncertainty(self, coefficients) -> Callable[[float], float]
```

Parâmetros

- `coefficients` (list ou `numpy.ndarray`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.

Retorno

- `uncertainty(t)` (Callable): Uma função que recebe um instante de tempo t e retorna a incerteza na posição $\sigma_x(t)$.

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função  $\sigma_x(t)$ 
sigma_x = p.position_uncertainty(coeffs)

# Avalia em diferentes instantes
print(sigma_x(0.0))    # → incerteza em t=0
print(sigma_x(0.5))    # → incerteza em t=0.5
print(sigma_x(1.0))    # → incerteza em t=1.0
```

Observações

- Internamente, a função utiliza `expected_position` e `expected_position_squared` para calcular $\langle x \rangle(t)$ e $\langle x^2 \rangle(t)$.
- A integração é feita numericamente via quadratura de Gauss com alta precisão.
- O uso de `max(aux1 - aux2**2, 0.0)` garante que erros numéricos não resultem em valores negativos dentro da raiz quadrada.
- Essa função é útil para estudar a dispersão espacial do pacote de onda e sua evolução temporal.
- Exceções
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`.
 - `RuntimeError`: Se as autofunções ou o espectro ainda não tiverem sido calculados (Ex.: `is_solved()` não foi executado).

POSITION_UNCERTAINTY_RELATIVE

Calcula a incerteza relativa na posição de um pacote de onda, definida como:

$$r_x(t) = \frac{\sigma_x(t)}{\langle x \rangle(t)}$$

onde:

- $\sigma_x(t)$ é a incerteza na posição,
- $\langle x \rangle(t)$ é o valor esperado da posição.

A função pode operar em dois modos:

1. **funcional:** se $t_{max} = \text{None}$, retorna a função $r_x(t)$ que pode ser avaliada em qualquer instante.
2. **análise:** se t_{max} for fornecido, avalia $r_x(t)$ no intervalo $[0, t_{max}]$ identifica os mínimos, salva os resultados em arquivo e plota o gráfico.

Assinatura

```
def position_uncertainty_relative(  
    self,  
    coefficients,  
    t_max: float = None,  
    num_points: int = 300,  
    save_filename: str = None  
) -> Union[Callable[[float], float], tuple]
```

Parâmetros

- `coefficients (list[complex])`: Lista de coeficientes da superposição de autofunções. Deve ter tamanho igual a `self.num_levels`.
- `t_max (float, opcional)`: Tempo máximo para avaliação. Se `None`, retorna apenas a função $r_x(t)$. Caso contrário, avalia $r_x(t)$ no intervalo $[0, t_{max}]$
- `num_points (int, padrão=300)`: Número de amostras de tempo no intervalo $[0, t_{max}]$.
- `save_filename (str, opcional)`: Nome do arquivo de saída para salvar os resultados. Se `None`, usa o padrão: "<root_filename>_min_relative_uncertainty.txt".

Retorno

- Se `t_max=None`:
 - `Callable[[float], float]`: a função $r_x(t)$.
- Caso contrário:
 - `tuple:(fig, min_val, min_times)`
 - `fig`: objeto `matplotlib.figure.Figure` com o gráfico.
 - `min_val`: valor mínimo de $r_x(t)$.
 - `min_times`: lista de instantes em que o mínimo é atingido.

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0, 0]

# (A) Obter a função r_x(t) e avaliar em t=2.5
r_fn = p.position_uncertainty_relative(coeffs)
print(f"r_x(2.5) = {r_fn(2.5):.4f}")

# (B) Avaliar em [0, 10], salvar resultados e plotar
fig, min_val, min_times = p.position_uncertainty_relative(
    coeffs,
    t_max=10.0,
    num_points=300,
    save_filename="min_relative_uncertainty.txt")
```

Observações

- Internamente, utiliza `expected_position` e `position_uncertainty` para calcular $\langle x \rangle(t)$ e $\sigma_x(t)$.
- Se $\langle x \rangle(t) = 0$, retorna ∞ .
- O arquivo salvo contém:
 - Valor mínimo de $r_x(t)$.
 - Instantes correspondentes.
 - Tabela completa com todos os pontos amostrados.
- O gráfico gerado destaca os pontos de mínimo com marcadores vermelhos.
- Exceções
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`.
 - `IOError`: Se ocorrer falha na escrita do arquivo de saída.
 - `RuntimeError`: Se as autofunções ou o espectro ainda não tiverem sido calculados (Ex.: `is_solved()` não foi executado).

EXPECTED_POSITION_IS_CALCULATED

Calcula e armazena em arquivo a evolução temporal do valor esperado da posição $\langle x \rangle(t)$ em um intervalo de tempo especificado.

A função gera um arquivo texto com duas colunas:

1. Tempo
2. Posição esperada

Assinatura

```
def expected_position_is_calculated(  
    self,  
    t_range: tuple[float, float],  
    coefficients: list[complex],  
    num_points: int = None  
) -> None
```

Parâmetros

- `t_range (tuple(float, float))`: Intervalo de tempo $[t_1, t_2]$ no qual a posição esperada será avaliada.
- `coefficients (list[complex])`: Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.
- `num_points (int, opcional)`: Número de pontos intermediários no intervalo de tempo.
 - Se fornecido, deve ser um inteiro positivo.
 - Se não fornecido, o padrão é 300 pontos.

Retorno

- `None`. Os resultados são gravados em um arquivo texto chamado: `<root_filename>_Position.txt`.

Exemplos de Uso

```
from PySpectral import SpectralMethod  
import numpy as np  
  
# Definição das funções f(x) e g(x)  
def f(x): return x**2  
def g(x): return 1  
  
# Instancia e resolve o problema  
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)  
p.is_solved()  
  
# Coeficientes da superposição  
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]
```

```
# Calcula  $\langle x \rangle(t)$  no intervalo [0, 5] com 200 pontos
p.expected_position_is_calculated((0, 5), coeffs, num_points=200)
```

Observações

- Internamente, utiliza a função `expected_position` para calcular $\langle x \rangle(t)$.
- O número de pontos gerados é `num_points + 1` (inclui os extremos t_1 e t_2).
- O arquivo é sobrescrito a cada execução.
- Cada linha do arquivo contém um par $(t, \langle x \rangle(t))$.
- Útil para análises numéricas e geração de dados para gráficos externos.
- Exceções
 - `ValueError`:
 - Se o número de coeficientes não corresponder a `self.num_levels`.
 - Se `num_points` não for um inteiro positivo.
 - `IOError`: Se ocorrer falha na escrita do arquivo de saída.
 - `RuntimeError`: Se as autofunções ou o espectro ainda não tiverem sido calculados (Ex.: `is_solved()` não foi executado).

EXPECTED_POSITION_AND_UNCERTAINTY_ARE_CALCULATED

Calcula e armazena em arquivo a evolução temporal do valor esperado da posição $\langle x \rangle(t)$ e sua incerteza $\sigma_x(t)$, gerando uma tabela com quatro colunas:

1. Tempo
2. Posição esperada $\langle x \rangle(t)$
3. Posição esperada – incerteza $\langle x \rangle(t) - \sigma_x(t)$
4. Posição esperada + incerteza $\langle x \rangle(t) + \sigma_x(t)$

Esse relatório auxilia na visualização da evolução e das margens de incerteza do pacote de ondas.

Assinatura

```
def expected_position_and_uncertainty_are_calculated(
    self,
    t_range: tuple[float, float],
    coefficients: list[complex],
    num_points: int = None
) -> None
```

Parâmetros

- `t_range(tuple(float, float))`: Intervalo de tempo $[t_1, t_2]$ no qual os valores serão avaliados.
- `coefficients (list[complex])`: Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.
- `num_points (int, opcional)`: Número de pontos intermediários no intervalo de tempo.
 - Se fornecido, deve ser um inteiro positivo.
 - Se não fornecido, o padrão é 300 pontos.

Retorno

- `None`. Os resultados são gravados em um arquivo texto chamado: `<root_filename>_PositionUncertainty.txt`

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]
```

```
# Calcula  $\langle x \rangle(t)$  e  $\sigma_x(t)$  no intervalo  $[0, 5]$  com 200 pontos
p.expected_position_and_uncertainty_are_calculated((0, 5), coeffs,
num_points=200)
```

Observações

- Internamente, utiliza as funções `expected_position` e `position_uncertainty`.
- O número de pontos gerados é `num_points + 1` (inclui os extremos t_1 e t_2).
- A função auxiliar `ifnotreal` garante que valores não reais sejam tratados como zero.
- O arquivo é sobrescrito a cada execução.
- Cada linha segue o formato: t , $\langle x \rangle$, $\langle x \rangle - \sigma_x(t)$ e $\langle x \rangle + \sigma_x(t)$, facilitando importação em planilhas ou scripts de pós-processamento.
- Útil para análises de dispersão temporal da posição e para gerar dados de gráficos com barras de incerteza.
- Exceções
 - `ValueError`:
 - Se o número de coeficientes não corresponder a `self.num_levels`.
 - Se `num_points` não for um inteiro positivo.
 - `IOError`: Se ocorrer falha na escrita do arquivo de saída.
 - `RuntimeError`: Se as autofunções ou o espectro ainda não tiverem sido calculados (Ex.: `is_solved()` não foi executado).

EXPECTED_POSITION_IS_PLOTTED

Lê um arquivo previamente gerado com os valores da posição esperada $\langle x \rangle(t)$ e produz um gráfico da evolução temporal.

O método procura automaticamente os arquivos de saída criados pelas funções:

- `expected_position_is_calculated()` → `<root_filename>_Position.txt`
- `expected_position_and_uncertainty_are_calculated()` → `<root_filename>_PositionUncertainty.txt`

Se encontrar um desses arquivos, plota a curva da posição esperada em função do tempo.

Assinatura

```
def expected_position_is_plotted(self) -> None
```

Parâmetros

- Nenhum parâmetro é exigido.
- Comportamento:
 1. Procura o arquivo `<root_filename>_Position.txt`.
 2. Se não existir, tenta `<root_filename>_PositionUncertainty.txt`.
 3. Lê os dados assumindo pelo menos duas colunas:
 - (1) tempo
 - (2) posição esperada
 4. Plota a curva em preto e salva a figura como:
`<root_filename>_ExpectedPosition.png`.

Retorno

- `None`. O método gera e exibe o gráfico, além de salvar a figura em arquivo PNG

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Primeiro, gera os dados da posição esperada
```

```
p.expected_position_is_calculated((0, 5), coeffs, num_points=200)

# Em seguida, plota os resultados
p.expected_position_is_plotted()
```

Observações

- O método depende de arquivos previamente gerados pelas funções de cálculo (`expected_position_is_calculated` ou `expected_position_and_uncertainty_are_calculated`).
- O gráfico é salvo automaticamente em alta resolução (PNG) e também exibido na tela.
- A curva é plotada em preto (`color='black'`) por padrão.
- Exceções
 - `FileNotFoundError`: Se nenhum dos arquivos esperados for encontrado.
 - `ValueError`: Se o arquivo existir, mas não contiver duas colunas válidas.
 - `RuntimeError`: Se ocorrer erro ao carregar ou interpretar os dados.

EXPECTED_POSITION_AND_UNCERTAINTY_ARE_PLOTTED

Lê um arquivo previamente gerado com os valores da posição esperada $\langle x \rangle(t)$ e sua incerteza $\sigma_x(t)$, e produz um gráfico mostrando a curva central e as faixas de incerteza.

O método espera encontrar o arquivo: `<root_filename>_PositionUncertainty.txt`.

Esse arquivo deve ter 4 colunas:

1. Tempo t
2. Posição esperada $\langle x \rangle(t)$
3. Posição esperada – incerteza $\langle x \rangle(t) - \sigma_x(t)$
4. Posição esperada + incerteza $\langle x \rangle(t) + \sigma_x(t)$

Assinatura

```
def expected_position_and_uncertainty_are_plotted(self) -> None
```

Parâmetros

- Nenhum parâmetro é exigido.
- Comportamento:
 1. Verifica se o arquivo `<root_filename>_PositionUncertainty.txt` existe.
 2. Lê os dados numéricos usando `numpy.loadtxt`.
 3. Se o arquivo tiver apenas 2 colunas, lança erro (pois não há dados de incerteza).
 4. Plota três curvas:
 - $\langle x \rangle(t) - \sigma_x(t)$ em cinza
 - $\langle x \rangle(t)$ em preto
 - $\langle x \rangle(t) + \sigma_x(t)$ em cinza
 5. Salva a figura como:
`<root_filename>_ExpectedPositionAndUncertainty.png`.
 6. Exibe o gráfico na tela.

Retorno

- `None`. O método gera e exibe o gráfico, além de salvar a figura em arquivo PNG

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()
```

```

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Primeiro, gera os dados da posição esperada com incerteza
p.expected_position_and_uncertainty_are_calculated((0, 5), coeffs,
num_points=200)

# Em seguida, plota os resultados
p.expected_position_and_uncertainty_are_plotted()

```

Observações

- O método depende do arquivo gerado por `expected_position_and_uncertainty_are_calculated`.
- O gráfico mostra a curva central $\langle x \rangle(t)$ e as curvas de limite superior e inferior ($\pm\sigma_x$).
- A região de incerteza pode ser destacada visualmente como uma “faixa” em torno da posição média.
- Exceções:
 - `FileNotFoundError`: Se o arquivo não existir.
 - `ValueError`: Se o arquivo não tiver 4 colunas
 - `RuntimeError`: Se ocorrer erro ao carregar ou interpretar os dados.

ANALYZE_EXPECTED_POSITION

Analisa os dados da posição esperada $\langle x \rangle(t)$ previamente calculados e armazenados em arquivo, identificando padrões como máximos, mínimos, período médio de oscilação, amplitude e valor médio.

Além disso, gera um relatório em arquivo texto e um gráfico com os pontos destacados

Assinatura

```
def analyze_expected_position(  
    self,  
    tolerance: float = 0.05,  
    save: bool = True,  
    show: bool = True  
) -> None
```

Parâmetros

- `tolerance` (float, padrão=0.05): Parâmetro reservado para ajustes de detecção de picos (não utilizado diretamente na versão atual).
- `save` (bool, padrão=True): Se True, salva o gráfico em arquivo PNG.
- `show` (bool, padrão=True): Se True, exibe o gráfico na tela.

Retorno

- None.

Comportamento:

- O método gera:
 - Um arquivo texto com os resultados da análise.
 - Um gráfico da posição esperada com máximos, mínimos e valor médio destacados.
- Arquivos Gerados
 1. **Entrada:**
 - `<root_filename>_Position.txt` (gerado por `expected_position_is_calculated`).
 2. **Saída:**
 - `<root_filename>_PositionAnalysis.txt` → relatório da análise.
 - `<root_filename>_ExpectedPosition_Analysis.png` → gráfico com marcações.

Exemplo de uso

```
from PySpectral import SpectralMethod  
import numpy as np  
  
# Definição das funções f(x) e g(x)  
def f(x): return x**2  
def g(x): return 1
```

```

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Primeiro, gera os dados da posição esperada
p.expected_position_is_calculated((0, 10), coeffs, num_points=400)

# Em seguida, analisa os resultados
p.analyze_expected_position()

```

Observações

- Utiliza `scipy.signal.find_peaks` para detectar máximos e mínimos locais.
- O período médio é estimado a partir dos intervalos entre máximos consecutivos.
- Se houver poucos picos, o período não pode ser estimado.
- O gráfico exibe:
 - Curva preta: $\langle x \rangle(t)$
 - Pontos vermelhos: máximos
 - Pontos azuis: mínimos
 - Linha verde tracejada: valor médio
- Exceções
 - `FileNotFoundError`: se o arquivo `<root_filename>_Position.txt` não existir.
 - `RuntimeError`: se houver erro na leitura do arquivo.

EXPECTED_MOMENTUM

Calcula numericamente o valor esperado do momento $\langle p \rangle(t)$ para uma função de onda $\psi(x, t)$ construída a partir de uma superposição de autofunções com coeficientes fornecidos.

Por definição:

$$\langle p \rangle(t) = \frac{\int_0^L \psi^*(x, t) \left(-i\hbar \frac{\partial \psi(x, t)}{\partial x} \right) w(x) dx}{\int_0^L |\psi(x, t)|^2 w(x) dx}$$

onde:

- L é o comprimento do domínio,
- $w(x)$ é a função peso definida no problema,
- $\hbar$ é a constante de Planck reduzida (aqui normalizada como $\hbar = 1$).

Assinatura

```
def expected_momentum(self, coefficients) -> Callable[[float], float]
```

Parâmetros

- `coefficients` (`list[complex]`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.

Retorno

- `expected_momentum_numeric(t)` (`Callable`): Uma função que recebe um instante de tempo t e retorna o valor esperado do momento $\langle p \rangle(t)$.

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função <p>(t)
exp_p = p.expected_momentum(coeffs)

# Avalia em diferentes instantes
print(exp_p(0.0))    # → valor esperado do momento em t=0
print(exp_p(0.5))    # → valor esperado do momento em t=0.5
```

```
print(exp_p(1.0))    # → valor esperado do momento em t=1.0
```

Observações

- A derivada espacial $\partial\psi/\partial x$ é aproximada numericamente por diferenças centrais (`np.gradient`).
- A integração é feita com a regra do trapézio (`np.trapz`).
- O denominador garante a normalização da função de onda.
- Se o denominador for zero (função de onda nula ou mal definida), é lançada uma exceção.
- O resultado pode ser complexo, refletindo componentes reais e imaginárias do momento esperado.
- Essa função é útil para estudar a dinâmica do momento médio de um pacote de onda.
- Exceções:
 - `ZeroDivisionError`: se o denominador da normalização for zero.
 - `ValueError`: Se o comprimento dos coeficientes (`coefficients`) for diferente do definido em `self.num_levels`.
 - `RuntimeError`: Se as autofunções ou o espectro de energia não estiverem calculados (`is_solved()` não foi chamado).
 - `TypeError`: Se os coeficientes não forem fornecidos como lista ou sequência válida.

MOMENTUM_UNCERTAINTY

Calcula numericamente a incerteza no momento $\Delta p(t)$ para uma função de onda $\psi(x, t)$ construída a partir de uma superposição de autofunções com coeficientes fornecidos.

A definição é:

$$\Delta p(t) = \sqrt{\langle p^2 \rangle(t) - \langle p \rangle(t)^2}$$

Onde $\langle p^2 \rangle(t)$ é calculado pela definição:

$$\langle p^2 \rangle(t) = \frac{\int_0^L \psi^*(x, t) \left(-\hbar^2 \frac{\partial^2 \psi(x, t)}{\partial x^2} \right) w(x) dx}{\int_0^L |\psi(x, t)|^2 w(x) dx}$$

e $\langle p \rangle(t)$ é calculado pela função `expected_momentum`.

Assinatura

```
def momentum_uncertainty(self, coefficients) -> Callable[[float], float]
```

Parâmetros

- `coefficients` (`list[complex]`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.

Retorno

- `momentum_uncertainty_numeric(t)` (`Callable`): Uma função que recebe um instante de tempo t e retorna a incerteza no momento $\Delta p(t)$.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função Δp(t)
delta_p = p.momentum_uncertainty(coeffs)

# Avalia em diferentes instantes
print(delta_p(0.0))    # → incerteza no momento em t=0
```

```
print(delta_p(0.5))    # → incerteza no momento em t=0.5
print(delta_p(1.0))    # → incerteza no momento em t=1.0
```

Observações

- A segunda derivada espacial $\partial^2\psi/\partial x^2$ é aproximada numericamente por diferenças centrais (`np.gradient` aplicado duas vezes).
- A integração é feita com a regra do trapézio (`np.trapz`).
- O denominador garante a normalização da função de onda.
- Se o denominador for zero (função de onda nula ou mal definida), é lançada uma exceção.
- Caso $\Delta p^2 \approx 0$, é emitido um aviso indicando que o estado pode ter momento bem definido.
- Essa função é complementar a `position_uncertainty`, permitindo explorar a relação de incerteza de Heisenberg:

$$\sigma_x \Delta p \geq \frac{\hbar}{2}$$

- Exceções
 - `ZeroDivisionError`: se o denominador da normalização for zero.

HEISENBERG_UNCERTAINTY

Calcula o produto das incertezas de posição e momento $\sigma_x(t)\sigma_p(t)$ e verifica se a desigualdade de Heisenberg é satisfeita:

$$\sigma_x(t) \sigma_p(t) \geq \frac{\hbar}{2}$$

onde:

- $\sigma_x(t)$ é a incerteza na posição,
- $\sigma_p(t)$ é a incerteza no momento,
- $\hbar$ é a constante de Planck reduzida (aqui normalizada como $\hbar = 1$).

Assinatura

```
def heisenberg_uncertainty(self, coefficients) -> Callable[[float], dict]
```

Parâmetros

- `coefficients` (`list[complex]`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.

Retorno

- `uncertainty_product(t)` (`Callable`): Uma função que recebe um instante de tempo t e retorna um dicionário com:
 - `"dx"`: incerteza na posição $\sigma_x(t)$
 - `"dp"`: incerteza no momento $\sigma_p(t)$
 - `"dx·dp"`: produto $\sigma_x(t)\sigma_p(t)$
 - `"hbar/2"`: limite teórico $\hbar/2$
 - `"valid"`: booleano indicando se a desigualdade é satisfeita

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Cria a função do produto de incertezas
heis = p.heisenberg_uncertainty(coeffs)
```

```

# Avalia em diferentes instantes
print(heis(0.0))
# Saída esperada (exemplo):
# {"σx": 0.2041,
#   "σp": 2.4512,
#   "σx · σp": 0.5002,
#   "ħ/2": 0.5,
#   "valid": True}

```

Observações

- Internamente, utiliza as funções `position_uncertainty` e `momentum_uncertainty`.
- O valor de $\hbar$ é assumido como 1.0 (unidades naturais).
- O campo `valid` indica se a desigualdade de Heisenberg é respeitada no instante avaliado.
- Essa função é útil para verificar numericamente a consistência da simulação com os princípios fundamentais da mecânica quântica.
- Exceções:
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`.
 - `ZeroDivisionError`: se a normalização da função de onda falhar em algum cálculo intermediário.

MOMENTUM_UNCERTAINTY_RELATIVE

Calcula a incerteza relativa no momento de um pacote de onda, definida como:

$$r_p(t) = \frac{\Delta p(t)}{\langle p \rangle(t)}$$

onde:

- $\Delta p(t)$ é a incerteza no momento,
- $\langle p \rangle(t)$ é o valor esperado do momento.

A função pode operar em dois modos:

1. **funcional**: se $t_{max} = \text{None}$, retorna à função $r_p(t)$ que pode ser avaliada em qualquer instante.
2. **análise**: se t_{max} for fornecido, avalia $r_p(t)$ no intervalo $[0, t_{max}]$ identifica os mínimos, salva os resultados em arquivo e plota o gráfico.

Assinatura

```
def momentum_uncertainty_relative(  
    self,  
    coefficients,  
    t_max: float = None,  
    num_points: int = 200,  
    save_filename: str = None  
) -> Union[Callable[[float], float], tuple]
```

Parâmetros

- `coefficients` (`list[complex]`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.
- `coefficients` (`list[complex]`): Lista de coeficientes da superposição de autofunções. Deve ter tamanho igual a `self.num_levels`.
- `t_max` (`float`, opcional): Tempo máximo para avaliação. Se `None`, retorna apenas a função $r_p(t)$. Caso contrário, avalia $r_p(t)$ no intervalo $[0, t_{max}]$.
- `num_points` (`int`, padrão=200): Número de amostras de tempo no intervalo $[0, t_{max}]$.
- `save_filename` (`str`, opcional): Nome do arquivo de saída para salvar os resultados. Se `None`, usa o padrão:
`<root_filename>_min_relative_momentum_uncertainty.txt`.

Retorno

- Se `t_max=None`:
 - `Callable[[float], float]`: a função $r_p(t)$.
- Caso contrário:

- tuple: (fig, min_val, min_times)
- fig: objeto matplotlib.figure.Figure com o gráfico.
- min_val: valor mínimo de $r_p(t)$.
- min_times: lista de instantes em que o mínimo é atingido.

Exemplo de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g)
p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0, 0]

# (A) Obter a função r_p(t) e avaliar em t=2.5
r_fn = p.momentum_uncertainty_relative(coeffs)
print(f"r_p(2.5) = {r_fn(2.5):.4f}")

# (B) Avaliar em [0, 10], salvar resultados e plotar
fig, min_val, min_times = p.momentum_uncertainty_relative(
    coeffs,
    t_max=10.0,
    num_points=300,
    save_filename="min_relative_momentum_uncertainty.txt")
```

Observações

- Internamente, utiliza `expected_momentum` e `momentum_uncertainty` para calcular $\langle p \rangle(t)$ e $\Delta p(t)$.
- Se $\langle p \rangle(t) = 0$, retorna ∞ .
- O arquivo salvo contém:
 - Valor mínimo de $r_p(t)$.
 - Instantes correspondentes.
 - Tabela completa com todos os pontos amostrados.
- O gráfico gerado destaca os pontos de mínimo com marcadores vermelhos.
- Exceções
 - `ValueError`: se o número de coeficientes não corresponder a `self.num_levels`.

EXPECTED_MOMENTUM_AND_UNCERTAINTY_ARE_CALCULATED

Calcula e armazena em arquivo a evolução temporal do momento esperado $\langle p \rangle(t)$ e sua incerteza $\sigma_p(t)$, gerando uma tabela com quatro colunas:

1. Tempo
2. Momento esperado $\langle p \rangle(t)$
3. Momento esperado – incerteza $\langle p \rangle(t) - \sigma_p(t)$
4. Momento esperado + incerteza $\langle p \rangle(t) + \sigma_p(t)$

Assinatura

```
def expected_momentum_and_uncertainty_are_calculated(
    self,
    t_range: tuple[float, float],
    coefficients: list[complex],
    num_points: int = None
) -> None
```

Parâmetros

- `t_range` (`tuple(float, float)`): Intervalo de tempo $[t_1, t_2]$ no qual os valores serão avaliados.
- `coefficients` (`list[complex]`): Lista de coeficientes da superposição de autofunções. O número de elementos deve ser igual a `self.num_levels`.
- `num_points` (`int`, opcional): Número de pontos intermediários no intervalo de tempo.
 - Se fornecido, deve ser um inteiro positivo.
 - Se não fornecido, o padrão é 300 pontos.

Retorno

- `None`. Os resultados são gravados em um arquivo texto chamado:
`<root_filename>_MomentumUncertainty.txt`

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
p.is_solved()
```

```
# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Calcula  $\langle p \rangle(t)$  e  $\sigma_p(t)$  no intervalo  $[0, 5]$  com 200 pontos
p.expected_momentum_and_uncertainty_are_calculated((0, 5), coeffs,
num_points=200)
```

Observações

- Internamente, utiliza as funções `expected_momentum` e `momentum_uncertainty`.
- O número de pontos gerados é `num_points + 1` (inclui os extremos t_1 e t_2).
- A função auxiliar `ifnotreal` garante que valores não reais sejam tratados como zero.
- O arquivo gerado segue o formato: $t, \langle p \rangle(t), \langle p \rangle(t) - \Delta p(t), \langle p \rangle(t) + \Delta p(t)$;
- O arquivo é sobrescrito a cada execução.
- Útil para análises de dispersão temporal do momento e para gerar dados de gráficos com barras de incerteza.
- Exceções
 - `ValueError`:
 - Se o número de coeficientes não corresponder a `self.num_levels`.
 - Se `num_points` não for um inteiro positivo.

EXPECTED_MOMENTUM_AND_UNCERTAINTY_ARE_PLOTTED

Lê um arquivo previamente gerado com os valores do momento esperado $\langle p \rangle(t)$ e sua incerteza $\sigma_p(t)$, e produz um gráfico mostrando a curva central e as curvas de limite superior e inferior.

O método espera encontrar o arquivo: `<root_filename>_MomentumUncertainty.txt`.

Esse arquivo deve ter 4 colunas:

1. Tempo, t .
2. Momento esperado, $\langle p \rangle(t)$.
3. Momento esperado – incerteza, $\langle p \rangle(t) - \sigma_p(t)$.
4. Momento esperado + incerteza, $\langle p \rangle(t) + \sigma_p(t)$.

Assinatura

```
def expected_momentum_and_uncertainty_are_plotted(self) -> None
```

Parâmetros

- Nenhum parâmetro é exigido.

Comportamento:

1. Verifica se o arquivo `<root_filename>_MomentumUncertainty.txt` existe.
2. Lê os dados numéricos usando `numpy.loadtxt`.
3. Se o arquivo não tiver exatamente 4 colunas, lança erro.
4. Plota três curvas:
 - $\langle p \rangle(t) - \sigma_p(t)$, em cinza.
 - $\langle p \rangle(t)$, em preto.
 - $\langle p \rangle(t) + \sigma_p(t)$, em cinza.
5. Salva a figura como:
`<root_filename>_ExpectedMomentumAndUncertainty.png`.
6. Exibe o gráfico na tela.

Retorno

- Nenhum (None) . O método gera e exibe o gráfico, além de salvar a figura em arquivo PNG

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Definição das funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia e resolve o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g)
```

```

p.is_solved()

# Coeficientes da superposição
coeffs = [1/np.sqrt(2), 1/np.sqrt(2), 0]

# Primeiro, gera os dados do momento esperado com incerteza
p.expected_momentum_and_uncertainty_are_calculated((0, 5), coeffs,
num_points=200)

# Em seguida, plota os resultados
p.expected_momentum_and_uncertainty_are_plotted()

```

Observações

- O método depende do arquivo gerado por `expected_momentum_and_uncertainty_are_calculated`.
- O gráfico mostra a curva central $\langle p \rangle(t)$ e as curvas de limite superior e inferior ($\pm\sigma_p$).
- A região de incerteza pode ser interpretada como a faixa de dispersão do momento médio ao longo do tempo.
- Exceções
 - `FileNotFoundError`: se o arquivo não existir.
 - `ValueError`: se o arquivo não tiver 4 colunas.
 - `RuntimeError`: se ocorrer falha ao gerar o gráfico.

ITS_NUMBER_OF_DIGITS

Retorna o número de dígitos atualmente utilizados pelo objeto.

Assinatura

```
def its_number_of_digits(self) -> int
```

Parâmetros

- Nenhum parâmetro é exigido. O método não recebe argumentos além de `self`.

Retorno

- `int`: O número de dígitos armazenado no atributo interno `self.digits_used`.

Exemplo de Uso

```
# Suponha que o objeto tenha sido configurado para usar 8 dígitos
print(p.its_number_of_digits())
# Saída: 8
```

Observações

- Esse método é apenas um `getter` para o atributo `digits_used`.
- Útil para verificar a precisão numérica configurada no objeto.
- Pode ser usado em conjunto com métodos que dependem da quantidade de dígitos para cálculos de alta precisão.

IS_DESCRIBED

Exibe na tela uma descrição detalhada do problema espectral configurado no objeto, incluindo parâmetros principais, funções utilizadas e estado atual do cálculo do espectro.

Assinatura

```
def is_described(self) -> None
```

Parâmetros

- Nenhum parâmetro é exigido.

Comportamento:

1. Mostra um resumo organizado com:
 - Número de níveis de energia (`num_levels`)
 - Comprimento do intervalo (`length`)
 - Representação das funções `f_function`, `g_function` e `weight`
 - Rótulo (`label`)
 - Número de dígitos configurados (`num_digits`)
2. Informa se o espectro de energia já foi calculado e se os autovetores foram obtidos.
3. Exibe o nome do “calculador” utilizado e a precisão em dígitos (`digits_used`).
4. Finaliza com um lembrete importante sobre o formato esperado para o potencial: $V_{\text{spectral}} = 2mV_{\text{real}}$

Retorno

- None: O método apenas imprime informações no console.

Exemplos de Uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x), g(x) e peso w(x)
def f(x): return x**2
def g(x): return 1
def w(x): return 1

# Instancia o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g,
w_function=w, label="Oscilador")

# Exibe a descrição do problema
p.is_described()

# Saída esperada (exemplo simplificado)
-----
Number of energy levels:      3
Interval amplitude (Length):  1.0 (float: 1.0)
f function (V_spectral):      x**2
```



```
g function: 1
w function (weight): 1
Label: Oscilador
Number of digits to be used: 50
The energy spectrum has not been calculated yet.
-----
Important: when using the spectral method, the potential must be expressed
as  $V_{\text{spectral}} = 2*m*V_{\text{real}}$ .
```

Observações

- A função auxiliar `get_func_repr` tenta extrair a expressão de retorno das funções `f_function`, `g_function` e `weight` para exibir de forma legível.
- Se não for possível extrair o código-fonte, exibe apenas o nome da função.
- Útil para documentar e revisar rapidamente a configuração do problema antes de rodar cálculos espectrais.

IS_DESCRIBED_TO_FILE

Escreve em um arquivo texto uma descrição detalhada do problema espectral configurado no objeto, incluindo parâmetros principais, funções utilizadas e estado atual do cálculo do espectro.

Assinatura

```
def is_described_to_file(self, filename: str) -> None
```

Parâmetros

- `filename (str)` : Nome do arquivo no qual a descrição será escrita.

Retorno

- `None`: O método apenas grava informações em arquivo e imprime uma mensagem de confirmação.

Exemplos de Uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x), g(x) e peso w(x)
def f(x): return x**2
def g(x): return 1
def w(x): return 1

# Instancia o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g,
w_function=w, label="Oscilador")

# Salva a descrição em arquivo
p.is_described_to_file("problem_description.txt")

# saída no console
Description has been written to file: problem_description.txt
```

Observações

- A função auxiliar `get_func_repr` retorna o nome da função (`__name__`) se disponível, ou a representação em string do objeto.
- Útil para documentar automaticamente a configuração do problema em arquivos de log ou relatórios.
- Complementa o método `is_described`, que imprime a descrição diretamente no console.
- Exceções:
 - `AttributeError`: Se o atributo `self.description` não estiver definido
 - `IOError` / `OSError`: Se ocorrer falha ao criar ou escrever no arquivo
 - `TypeError`: Se `filename` não for uma string válida

COMPARE_NUMERICALLY

Compara numericamente os espectros de energia (autovalores) de dois problemas espectrais resolvidos, o problema chamador (`self`) e outro problema (`other`). O método gera um arquivo texto contendo os autovalores de ambos os problemas, bem como as variações absolutas e percentuais entre eles.

Assinatura

```
def compare_numerically(self, other) -> None
```

Parâmetros

- `other` (`SpectralMethod`): Outra instância da classe, representando um problema espectral a ser comparado com o problema chamador.

Retorno

- `None`. O método apenas gera o arquivo de comparação e imprime mensagens de confirmação no console

Exemplo de Uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia dois problemas espectrais
p1 = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g,
label="Problema A")
p2 = SpectralMethod(num_levels=3, length=1.2, f_function=f, g_function=g,
label="Problema B")

# Resolve ambos
p1.is_solved()
p2.is_solved()

# Compara numericamente os espectros
p1.compare_numerically(p2)

# saída no console
Comparison of spectra has been written in the file
ProblemA_Comparison_Eigenvalues.txt
Done
```

Observações

- O cálculo da variação percentual evita divisão por zero usando `math.isclose`.
- Apenas os primeiros `min(len(pSp), len(qSp))` autovalores são comparados.

- O arquivo gerado tem o nome: `<root_filename>_Comparison_Eigenvalues.txt`. E contém:
 - Cabeçalho com informações do programa e dos problemas.
 - Tabela com colunas:
 - Nível
 - Autovalor do problema chamador
 - Autovalor do problema comparado
 - Variação absoluta (comparado – chamador)
 - Variação percentual (relativa ao autovalor do chamador)
- Útil para comparar diferentes potenciais, condições de contorno ou parâmetros e analisar como o espectro de energia se altera.
- Exceções:
 - `AttributeError`: Se `self.spectrum` ou `other.spectrum` não existirem.
 - `IOError` / `OSError`: Se ocorrer falha na criação ou escrita do arquivo.
 - `ValueError`: Se os espectros tiverem comprimentos diferentes e não puderem ser comparados nível a nível.

COMPARE_GRAPHICALLY

Compara graficamente os autovalores (e autovetores) de dois problemas espectrais resolvidos, o problema chamador (`self`, `qSp`) e outro problema (`other`, `pSp`).

O método gera três tipos de gráficos:

1. Variações absolutas ($\log_{10}$) dos autovalores:
 - Azul: diferenças positivas ($qSp - pSp > 0$)
 - Vermelho: diferenças negativas ($qSp - pSp < 0$)
 - Verde: diferenças nulas
2. Variações percentuais dos autovalores: $\frac{qSp-pSp}{|pSp|} \times 100$
3. Erros de ortogonalidade dos autovetores (produto escalar entre autovetores distintos), em escala logarítmica.

Assinatura

```
def compare_graphically(self, other) -> None
```

Parâmetros

- `other` (`SpectralMethod`) : Outra instância da classe, representando um problema espectral a ser comparado com o problema chamador.

Retorno

- `None`: O método apenas exibe gráficos e imprime informações no console

Exemplo de uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x) e g(x)
def f(x): return x**2
def g(x): return 1

# Instancia dois problemas espectrais
p1 = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g,
label="Problema A")
p2 = SpectralMethod(num_levels=4, length=1.2, f_function=f, g_function=g,
label="Problema B")

# Resolve ambos
p1.is_solved()
p2.is_solved()

# Compara graficamente os espectros
p1.compare_graphically(p2)
```

Observações

- O número de níveis comparados é o mínimo entre os dois conjuntos de autovalores.
- As diferenças são classificadas em positivas, negativas ou nulas para melhor visualização.
- O gráfico de ortogonalidade mostra o desvio do produto escalar entre autovetores distintos.
- A escala logarítmica simétrica (`symlog`) é usada para capturar variações muito pequenas.
- Útil para analisar visualmente como diferentes potenciais, condições de contorno ou parâmetros afetam o espectro e a ortogonalidade dos autovetores.
- Exceções
 - o `AttributeError`: Se `self.spectrum` ou `other.spectrum` não estiverem definidos.
 - o `ValueError`: Se houver divisão por zero ao calcular variação percentual (autovalor igual a zero).

CHECK_SOLUTIONS_NUMERICALLY

Verifica numericamente a consistência das soluções obtidas (autovalores e autovetores) de um problema espectral, de acordo com três critérios principais:

1. Erro da equação: Substitui cada par (E, v) na equação $M(E)v = (FirstBigMatrix() - E \cdot SecondBigMatrix())v = 0$ e calcula o máximo valor absoluto dos elementos resultantes.
2. Desvio de normalidade: Calcula $|1 - \langle v, v \rangle|$ para cada autovetor, verificando se está devidamente normalizado.
3. Desvio de ortogonalidade: Para cada par de autovetores (v_i, v_j) , calcula o produto escalar $\langle v_i, v_j \rangle$. Os resultados são salvos em arquivos texto separados para análise posterior.

As funções `FirstBigMatrix()`, `SecondBigMatrix()` são internas, relacionadas as matrizes do método espectral.

Assinatura

```
def check_solutions_numerically(self) -> None
```

Parâmetros

- Nenhum parâmetro é exigido.

Arquivos Gerados:

1. `<root_filename>_Check_Equations.txt`
 - Contém os erros da equação $M(E)v = 0$.
 - Colunas:
 - Nível de energia
 - Erro máximo absoluto
2. `<root_filename>_Check_Normality.txt`
 - Contém os desvios de normalidade dos autovetores.
 - Colunas:
 - Nível de energia
 - Erro $|1 - \langle v, v \rangle|$
3. `<root_filename>_Orthogonality.txt`
 - Contém os desvios de ortogonalidade entre pares de autovetores.
 - Colunas:
 - Nível i
 - Nível j
 - Erro $\langle v_i, v_j \rangle$ (apenas parte real é registrada)

Retorno

- `None`. O método apenas gera arquivos de saída e imprime mensagens de progresso no console.

Exemplo de Uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x), g(x) e peso w(x)
def f(x): return x**2
def g(x): return 1
def w(x): return 1

# Instancia o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g,
w_function=w, label="Oscilador")

# Resolve o problema espectral
p.is_solved()

# Verifica numericamente as soluções
p.check_solutions_numerically()

# saída no console
Reporting errors in equations in file: Oscilador_Check_Equations.txt
Reporting errors in eigenvector normality in file:
Oscilador_Check_Normality.txt
Reporting errors in eigenvector orthogonality in file:
Oscilador_Check_Orthogonality.txt
Done.
```

Observações

- O espectro (autovalores e autovetores) deve ter sido previamente calculado.
- Se algum produto escalar entre autovetores resultar em número complexo, apenas a parte real é registrada, e um aviso é exibido no console.
- Os erros são gravados em notação científica com alta precisão (20 casas decimais).
- Útil para auditar a qualidade numérica das soluções obtidas, especialmente em cálculos de alta precisão.
- Exceções:
 - `AttributeError`: Se `its_eigenvectors`, não estiver definido.
 - `ValueError`: Se dimensões de matrizes e vetores forem incompatíveis.
 - `IOError` / `OSError`: Se houver falha na criação ou escrita dos arquivos de saída.

CHECK_SOLUTIONS_GRAPHICALLY

Verifica graficamente a qualidade dos autovalores e autovetores obtidos, avaliando três critérios principais:

1. **Erro de substituição:** Mede o erro ao substituir (E, v) na equação

$$M(E)v = (FirstBigMatrix() - E \cdot SecondBigMatrix())v = 0$$

e plota $\log_{10}(|M(E)v|)$ para cada nível de energia.

2. **Desvio de normalidade:** Mede $|1 - \langle v, v \rangle|$ para cada autovetor, verificando se estão normalizados, e plota $\log_{10}(|1 - \langle v, v \rangle|)$.
3. **Desvio de ortogonalidade:**
 - Resumo 1D: para cada nível, mostra o pior desvio de ortogonalidade em relação aos demais.
 - Resumo 2D: mapa de calor (scatter plot colorido) com $\log_{10}(|\langle v_i, v_j \rangle|)$ para cada par de autovetores.

As funções `FirstBigMatrix()`, `SecondBigMatrix()` são internas, relacionadas as matrizes do método spectral.

Se os arquivos de verificação (`*_Check_Equations.txt`, `*_Check_Normality.txt`, `*_Check_Orthogonality.txt`) já existirem, os dados são lidos deles. Caso contrário, os cálculos são refeitos.

Assinatura

```
def check_solutions_graphically(self, save: bool = True, show: bool = True)
-> None
```

Parâmetros

- `save (bool, padrão=True)`: Se `True`, salva os gráficos em arquivos PNG.
- `show (bool, padrão=True)`: Se `True`, exibe os gráficos na tela. Se `False`, fecha as figuras após salvar.

Retorno

- `None`. O método apenas gera e exibe/salva gráficos

Exemplos de Uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x), g(x) e peso w(x)
def f(x): return x**2
def g(x): return 1
def w(x): return 1

# Instancia o problema
p = SpectralMethod(num_levels=4, length=1.0, f_function=f, g_function=g,
w_function=w, label="Oscilador")
```

```
# Resolve o problema espectral
p.is_solved()

# Verifica graficamente a qualidade das soluções
p.check_solutions_graphically()
```

Observações

- Se os arquivos de verificação não existirem, os cálculos são refeitos diretamente a partir dos autovalores e autovetores.
- Os erros são representados em escala logarítmica ($\log_{10}$) para melhor visualização.
- O gráfico 2D de ortogonalidade usa coloração (`cmap="viridis"`) para indicar a magnitude do erro.
- Arquivos Gerados
 - `<root_filename>_Check_Substitution.png`
 - `<root_filename>_Check_Normality.png`
 - `<root_filename>_Check_Orthogonality_Summary.png`
 - `<root_filename>_Check_Orthogonality.png`
- Útil para auditar visualmente a qualidade numérica das soluções obtidas.
- Exceções
 - `AttributeError`: Se os métodos ou arquivos de dados não estiverem disponíveis.
 - `RuntimeError`: Se ocorrer falha ao computar os dados ou gerar os gráficos.
 - `ValueError`: Se os dados lidos não estiverem no formato esperado (colunas ou dimensões incorretas).

CHECK_EQUATIONS

Verifica numericamente se cada par autovalor–autovetor (E, v) satisfaz aproximadamente a equação de autovalores generalizada:

$$Mv = (F - E \cdot S)v \approx 0$$

onde:

- F é a matriz obtida pela função interna `build_matrix_d()`,
- S é a matriz obtida pela função interna `build_matrix_d_prime()`.

O método calcula o erro máximo absoluto dos componentes de $(F - E \cdot S)v$ para cada nível de energia e salva os resultados em um arquivo texto. O desvio da normalidade dos autovetores é verificado pela avaliação do produto escalar de cada autovetor consigo mesmo: $|1 - v \cdot v|$. Neste cálculo, a função peso é levada em consideração pelo procedimento interno da biblioteca `ScalarProduct`.

O mesmo número de dígitos usado no cálculo do espectro é usado nesta comparação. Os erros máximos são reportados em arquivo de texto.

Assinatura

```
def check_equations(self, filename: str) -> None
```

Parâmetros

- `Filename (str)`: Nome do arquivo de saída onde os resultados serão gravados.

Retorno

- `None`. O método apenas gera o arquivo de saída e imprime mensagens no console.

Exemplo de Uso

```
from PySpectral import SpectralMethod

# Definição de funções f(x), g(x) e peso w(x)
def f(x): return x**2
def g(x): return 1
def w(x): return 1

# Instancia o problema
p = SpectralMethod(num_levels=3, length=1.0, f_function=f, g_function=g,
w_function=w, label="Oscilador")

# Resolve o problema espectral
p.is_solved()

# Verifica numericamente a equação de autovalores
p.check_equations("Oscilador_Check_Equations.txt")
```

Observações

- Pequenos erros residuais são esperados devido a aproximações numéricas.

- O parâmetro de tolerância padrão é 10^{-15} . Valores abaixo disso são tratados como zero.
- Complementa os métodos `check_solutions_numerically` e `check_solutions_graphically`.
- O método é útil para auditar a precisão numérica das soluções obtidas.
- Exceções:
 - `RuntimeError`: Se as soluções (espectro ou vetores) ainda não tiverem sido calculadas.
 - `IOError` / `OSError`: Se houver falha ao abrir ou escrever no arquivo especificado.

MYDENSITYPLOT

Gera um gráfico de densidade estilizado ("cartoon") para uma função auxiliar $aux(x)$ definida no intervalo $[a, b]$. A função é avaliada em subdivisões do intervalo, normalizada e transformada em uma linha de imagem. Essa linha é replicada verticalmente para formar uma imagem 2D, que é exibida como um mapa de densidade.

Assinatura

```
def mydensityplot(
    self,
    aux: Callable[[float], float],
    x_label: str,
    a: float,
    b: float,
    N: int = 200,
    cmap: str = "gray_r"
) -> tuple[matplotlib.figure.Figure, matplotlib.axes.Axes]
```

Parâmetros

- `aux (callable)`: Função que recebe um valor x e retorna um valor numérico.
- `x_label (str)`: Rótulo do eixo x .
- `a (float)`: Limite inferior do intervalo.
- `b (float)`: Limite superior do intervalo.
- `N (int, opcional, padrão=200)`: Número de subdivisões do intervalo.
- `cmap (str, opcional, padrão="gray_r")`: Colormap do Matplotlib a ser usado na visualização.

Retorno

- `fig, ax`: Objetos `matplotlib.figure.Figure` e `matplotlib.axes.Axes` correspondentes ao gráfico gerado.

Exemplo de Uso

```
import numpy as np
from PySpectral import SpectralMethod

# Função auxiliar: uma gaussiana
def gaussian(x):
    return np.exp(-((x-0.5)**2) / 0.01)

# Instancia o problema (apenas para usar o método de plotagem)
p = SpectralMethod(num_levels=3, length=1.0, f_function=lambda x: x,
g_function=lambda x: 1)

# Gera o gráfico de densidade no intervalo [0,1]
fig, ax = p.mydensityplot(gaussian, x_label="x", a=0.0, b=1.0, N=300,
cmap="viridis")
```

Observações

- Se a função `aux` for constante, o denominador da normalização é ajustado para evitar divisão por zero.
- O parâmetro `height` (fixo em 50) controla quantas vezes a linha é replicada verticalmente.
- O eixo vertical não tem significado físico direto — é apenas uma dimensão auxiliar para construir a imagem.
- Útil para criar representações visuais rápidas de funções de onda, densidades de probabilidade ou potenciais.

OPTIMIZE_LENGTH

Determina o valor ótimo de L (comprimento do domínio) para a equação de Schrödinger unidimensional resolvida pelo método espectral. A ideia é encontrar o L que minimiza a energia do estado fundamental (menor autovalor do Hamiltoniano), recorrendo a um método melhorado de procura dentro de limites predefinidos.

Assinatura

```
def optimize_length(self, num_levels_opt: int = None, L_bounds: tuple = (0.1, 50.0), verbose: bool = False) -> float
```

Parâmetros

- `num_levels_opt` (int, padrão=None): Número de funções de base (níveis) a considerar na otimização. Se for None, utiliza o valor padrão (`self.num_levels`) definido na instância.
- `L_bounds` (tuple, padrão=(0.1, 50.0)): Tupla contendo os limites (inferior e superior) para a procura do comprimento ótimo.
- `verbose` (bool, padrão=False): Se True, imprime no console informações detalhadas sobre os limites e os resultados intermédios da otimização.

Retorno

- `L_optimal` (float): valor de L que minimiza a energia do estado fundamental.
- `E0` (float): Energia do estado fundamental (menor autovalor do Hamiltoniano) correspondente ao comprimento ótimo (L ótimo).

Exemplos de Uso

```
from PySpectral import SpectralMethod
import numpy as np

# Potencial harmônico
def V(x): return 0.5 * x**2

# Peso
def w(x): return 1

# Instancia o problema
p = SpectralMethod(num_levels=10, length=1.0, f_function=V,
                  g_function=lambda x: 1, w_function=w)

# Determina o L ótimo com impressão de progresso (verbose)
L_opt = p.optimize_length(num_levels_opt=10, L_bounds=(0.1, 50.0),
                          verbose=True)

print(f"L ótimo = {L_opt:.4f}")
```


Observações

- O método limpa as *caches* internas (`self.clear_caches()`) no início da execução para garantir que as matrizes sejam recalculadas corretamente com os novos valores de L .
- A escolha apropriada dos limites em `L_bounds` e do número de níveis (`num_levels_opt`) influencia diretamente na convergência, precisão e desempenho do cálculo.
- A matriz cinética é construída analiticamente, enquanto a matriz de potencial é avaliada numericamente por quadratura de Gauss.
- O método é útil para ajustar automaticamente o domínio em problemas de poços quânticos ou potenciais confinantes, garantindo melhor precisão numérica.
- A função está envolvida num decorador `@safe_execution` que fará a gestão de eventuais exceções e erros operacionais durante a execução do processo.
- **Exceções**
 - `ValueError`: Se os valores definidos na tupla `L_bounds` forem inválidos (por exemplo, limite inferior igual ou superior ao limite superior).
 - `RuntimeError`: Se a otimização falhar na convergência para um valor de comprimento L estável dentro dos limites fornecidos.

EXEMPLOS COMPLETOS

Half Oscillator

O oscilador quântico meio-harmônico é um bom exemplo porque mantém a estrutura simples do potencial harmônico, mas ao ser limitado ao semieixo positivo introduz uma barreira infinita em $x = 0$. Essa modificação gera condições de contorno diferentes, alterando o espectro de energia e as autofunções. Por isso, ele ilustra bem como mudanças nas fronteiras afetam um sistema quântico e serve como modelo acessível para testar métodos numéricos e conceitos fundamentais da mecânica quântica [5].

Dessa forma, para servir de exemplo dos principais métodos, vamos utilizar a solução da equação de Schrödinger para o oscilador quântico meio-harmônico [6]:

$$\left(-\frac{d^2}{dx^2} + kx^2\right)\Psi(x) = E\Psi(x, t), \quad x \geq 0 \quad (24)$$

em que k é uma constante positiva. A função $\psi(x)$ representa a parte espacial da função de onda de um sistema quântico, e E é o autovalor de energia.

Da equação acima, podemos definir as funções $f(x) = kx^2$ e $g(x) = 1$. Além disto, para este exemplo, utilizaremos $k = 1$, no intervalo $x \in [0, 10]$ com 100 níveis, e atribuiremos o resultado à variável q .

Utilizando a biblioteca `PySpectral`, podemos definir o problema “*Half Oscillator*” pelo comando:

```
Comando # função 'f' - define o potencial do problema.
def f(x):
    return x**2

# função 'g'.
def g(x):
    return 1

# função 'w' - define a função peso do problema.
def w(x):
    return 1

# definindo o objeto
q = SpectralMethod(
    num_levels=100,
    length=10,
    f_function=f,
    g_function=g,
    weight=w,
    num_digits=15,
    label="HalfOscilattor",
    root_filename=' HalfOscilattor ',
    optimizer_L='n')
```

Dessa maneira, criamos uma instancia `q` da classe espectral, podendo utilizar os métodos disponíveis para a classe. Podemos imprimir na tela as informações armazenadas no objeto, para validação. Para isso, utilizamos o comando:

Comando `q.is_described()`

Saída -----
Number of energy levels: 100
Interval amplitude (Length): 10 (float: 10.0)
f function (V_spectral): x^2
g function: 1
w function (weight): 1
Label: HalfOscilattor
Number of digits to be used: 15
The energy spectrum has not been calculated yet.

Important: when using the spectral method, the potential must be expressed
as $V_{\text{spectral}} = 2mV_{\text{real}}$.

Com as informações do problema checadas, podemos dar sequência. Agora, a solução pode ser calculada através da execução do seguinte método:

Comando `q.is_solved()`

Saída Eigenvalues already calculated. Skipping eigenvalue calculation.
Calculating eigenvectors ...

Calculating energy eigenvectors now ...
Eigenvectors saved to: HalfOscilattor_Eigenvectors.txt

Done!

=====

Time elapsed for eigenvector calculation: 11.362255 seconds.

=====

The eigenvalue/eigenvector problem has been completely solved with 15
digits used.

Em seguida, podemos verificar os 10 autovalores de energia mais baixa do sistema, através do comando:

Comando `q.its_eigenvalues(slice(1, 11))`

```
Saída [np.float64(2.99999999999999485),
        np.float64(7.0000000000000011),
        np.float64(10.9999999999999908),
        np.float64(14.999999999999981),
        np.float64(18.999999999999967),
        np.float64(22.999999999999985),
        np.float64(27.000000000000004),
        np.float64(30.999999999999936),
        np.float64(34.99999999999992),
        np.float64(38.99999999999993)]
```

A previsão analítica para os autovalores de energia E_n desse potencial é bem conhecida [6]:

$$E_n = 2(n + 1), \quad n \in \{1, 3, 5, \dots\}$$

Dessa forma, os 10 menores níveis a partir dessa expressão são:

$$E_n = [3, 7, 11, 15, 19, 23, 27, 31, 35, 39]$$

Concordando com a previsão numérica da biblioteca.

Dando continuidade, podemos calcular o valor esperado da posição $\langle x(t) \rangle$, através do método `expected_position`. Nesse caso, é necessário passar uma lista com os coeficientes que representam a combinação linear da autofunção. Considerando uma superposição equiprovável dos três primeiros níveis, podemos definir os coeficientes de acordo com a sintaxe da linguagem Python e utilizar o método, de acordo com o exemplo:

```
Comando coefficients = [1,1,1]+[0]*97
```

```
Av = q.expected_position(coefficients = coefficients)
```

Dessa forma, a variável, A_v passa a ser uma função do tempo t . Logo, o valor esperado para a posição $\langle x(t) \rangle$ no tempo $t = 0$, pode ser obtido fazendo:

```
Comando Av(0)
```

```
Saída 2.304509980410253
```

A incerteza para o fator de escala em função do tempo é obtida através do método `position_uncertainty`:

```
Comando coefficients = [1,1,1]+[0]*97
```

```
Un = q.position_uncertainty(coefficients)
```

Novamente, passamos os coeficientes para o método. Assim, a incerteza associada posição no instante de tempo $t = 0$ é calculada fazendo:

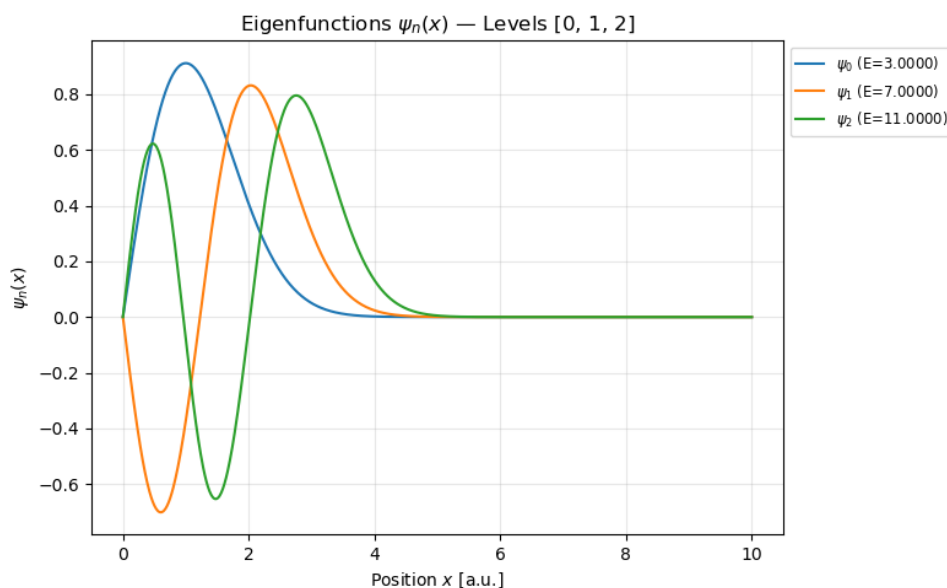
Comando `Un(0)`

Saída 0.7045866278301305

Outra grandeza relevante na análise do problema são as funções de onda. Utilizando o método `p.plot_eigenfunctions(num_levels=3)` geramos o gráfico para as funções de onda dos primeiros três níveis.

Comando `p.plot_eigenfunctions(num_levels=3)`

Saída Eigenpairs saved in: HalfOscilattor/HalfOscilattor_Eigenpairs.txt
--- Processing 3 levels---
-> Saved TXT: level 0
-> Saved TXT: level 1
-> Saved TXT: level 2
-> Image saved: HalfOscilattor/HalfOscilattor_eigenfunctions.png



A unitariedade da dinâmica foi verificada por meio da conservação da norma da função de onda. Para essa verificação, utilizou-se o método `norm_of_wave_function`, responsável por calcular a norma da função de onda em função do tempo, permitindo avaliar seu comportamento em diferentes instantes e confirmar a consistência física da solução. Esse método retorna uma função dependente do tempo t ; portanto, a variável que armazena o resultado passa a ser uma função. Dessa forma, é possível avaliar a função obtida por meio

de um procedimento iterativo, verificando a norma em diferentes instantes e assegurando a preservação da unitariedade.

```
Comando n = q.norm_of_wave_function(coefficients)
for t in np.linspace(0, 10, 11):
    print(f"Norm at t= {t:.1f}: {n(t):.15}")
```

```
Saída Norm at t= 0.0: 0.9999999999999968
Norm at t= 1.0: 1.0000000000000002
Norm at t= 2.0: 0.9999999999999999
Norm at t= 3.0: 0.9999999999999967
Norm at t= 4.0: 1.0000000000000005
Norm at t= 5.0: 0.9999999999999997
Norm at t= 6.0: 0.9999999999999997
Norm at t= 7.0: 1.0000000000000005
Norm at t= 8.0: 0.9999999999999966
Norm at t= 9.0: 0.9999999999999989
Norm at t= 10.0: 1.0000000000000002
```

Para a análise dentro de um intervalo de tempo maior, o procedimento `expected_position_and_uncertainty_are_calculated` pode ser utilizado. Este método calcula simultaneamente a posição e a incerteza associada dentro do intervalo de tempo solicitado, salvando o resultado em um arquivo txt (para o exemplo, com o nome: `HalfOscilattor_PositionUncertainty.txt`). Por padrão, é calculado 300 pontos dentro do intervalo, mas esse valor pode ser alterado. No nosso exemplo, vamos considerar apenas 100 pontos. Após a execução do comando, um aviso indicará que o arquivo com os resultados foi criado e salvo:

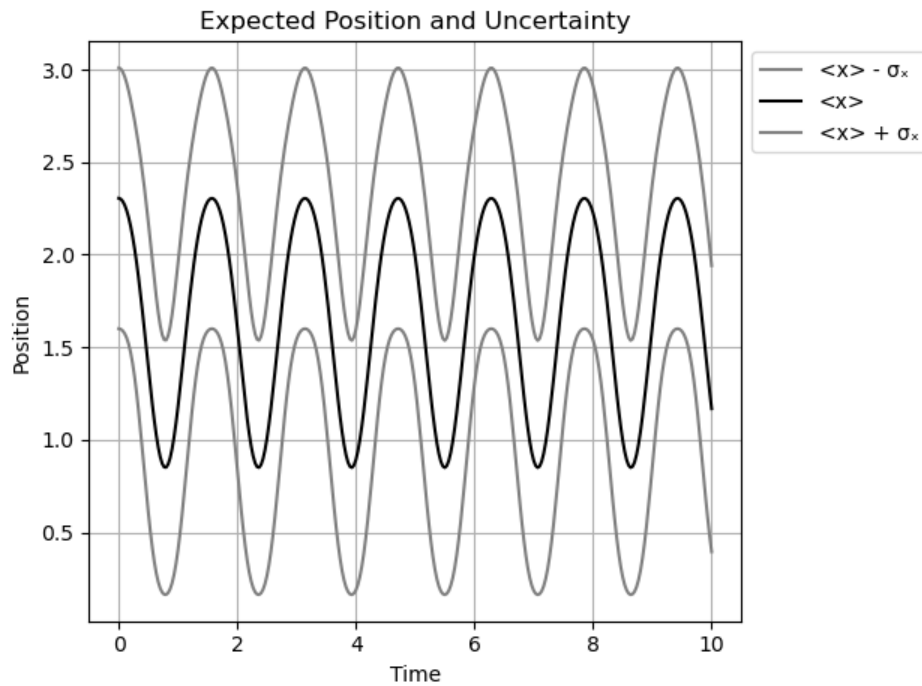
```
Comando coefficients = [1,1,1]+[0]*97 # coeficientes
dt = (0, 10) # intervalo de tempo desejado
q.expected_position_and_uncertainty_are_calculated(t_range=dt,
coefficients=coefficients, num_points=300)
```

```
Saída Expected position and uncertainty data stored in file:
HalfOscilattor_PositionUncertainty.txt
```

Realizado esse procedimento, os resultados podem ser visualizados graficamente. Para isso, o método `expected_position_and_uncertainty_are_plotted()` precisa ser executado. Este procedimento, lê os dados do arquivo txt (gerado pela função anterior) e gera um gráfico da posição com as incertezas, que é apresentado. Seu comando é dado por:

Comando `q.expected_position_and_uncertainty_are_plotted( )`

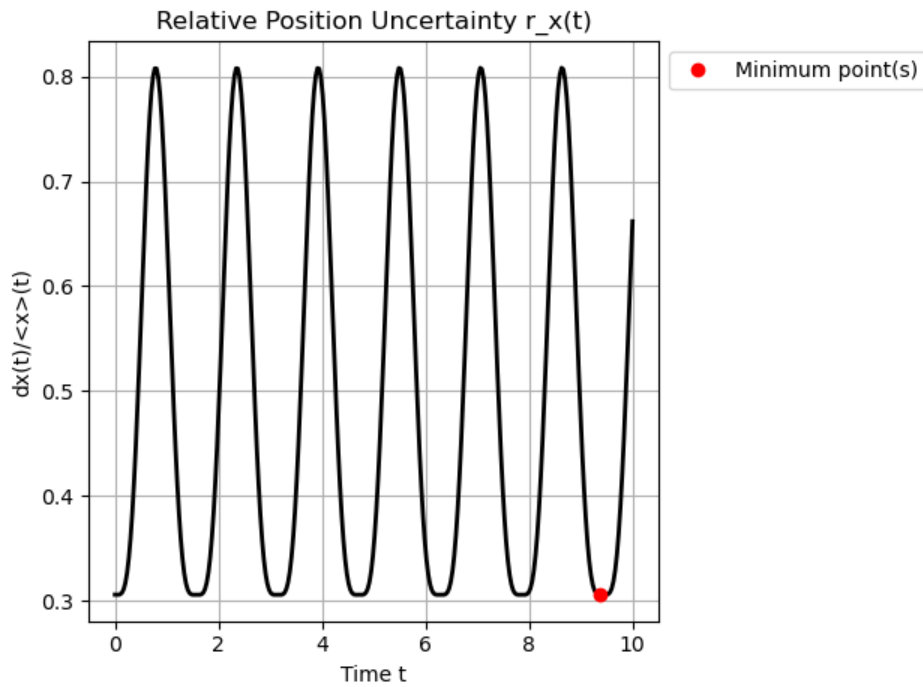
Saída



Podemos também analisar a incerteza relativa à posição em função do tempo. Para isso, executamos o comando `position_uncertainty_relative()`. Ele retorna, o menor valor encontrando no intervalo considerado, o período associado (quando houver tendência oscilatória) e imprime o gráfico na tela. A seguir está o exemplo da execução desse comando:

Comando `tmax = 10
points = 300
q.position_uncertainty_relative(coefficients, t_max=tmax,
num_points=points)`

Saída Minimum value of $dx(t) / \langle x \rangle(t)$: 0.305643215383337
Estimated period(s):
`t = 9.364548494983277`
Full results saved at:
`HalfOscilattor/HalfOscilattor_min_relative_uncertainty.txt`



Além dos dados obtidos para a posição esperada, é possível calcular e analisar o comportamento do momento esperado associado ao problema espectral em estudo. De maneira análoga, o momento pode ser determinado pelo método:

Comando `P_t = q.expected_momentum(coefficients=coefficients)`
`P_t(1)`

Saída 1.3726150598523958

O método `expected_momentum()`, retorna uma função. No exemplo apresentado, a variável `P_t` é definida como função do tempo. Dessa forma, para cada valor de tempo fornecido, a função retorna o momento correspondente ao respectivo instante. Além disso, é possível calcular a incerteza associada por meio da seguinte expressão:

Comando `dp = q.momentum_uncertainty(coefficients)`
`dp(1)`

Saída 1.766205227286292

A partir deste ponto, torna-se possível calcular e gerar os gráficos referentes ao momento e às incertezas associadas. Para tal finalidade, emprega-se inicialmente o método `expected_momentum_and_uncertainty_are_calculated()`, responsável por realizar o cálculo do momento e das respectivas incertezas, armazenando os resultados em um arquivo de texto. A execução do método pode ser realizada conforme o seguinte procedimento:

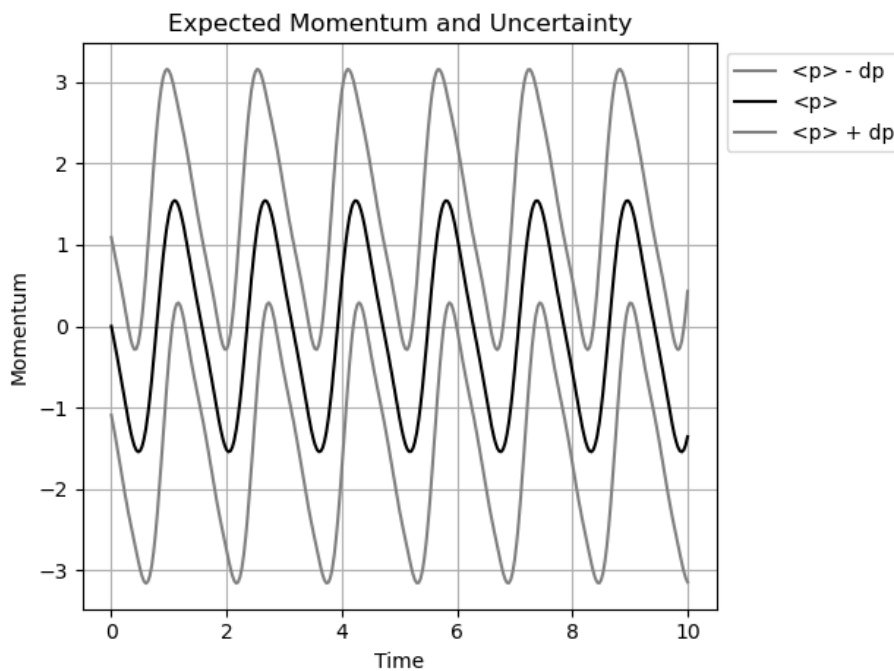
Comando `dt = (0.0, 10) # intervalo de tempo desejado`
`points = 300 # número de pontos gerados`
`q.expected_momentum_and_uncertainty_are_calculated(dt, coefficients,`
`points)`

Saída Expected momentum and uncertainty data stored in file:
HalfOscillator_MomentumUncertainty.txt

A partir dos valores armazenados no arquivo de texto, torna-se possível visualizar graficamente os resultados obtidos por meio do método `expected_momentum_and_uncertainty_are_plotted()`. Esse procedimento permite representar tanto o comportamento do momento quanto as incertezas associadas ao problema em análise.

Comando `q.expected_momentum_and_uncertainty_are_plotted()`

Saída



Para essa grandeza, é possível explorar a razão entre a incerteza e o momento. Nesse contexto, emprega-se o método `momentum_uncertainty_relative()`, o qual permite calcular essa relação de forma sistemática. A execução pode ser realizada conforme o procedimento a seguir:

Comando `tmax = 10`
`points = 300`
`p.momentum_uncertainty_relative(coefficients, t_max=tempo,`
`num_points=pontos)`

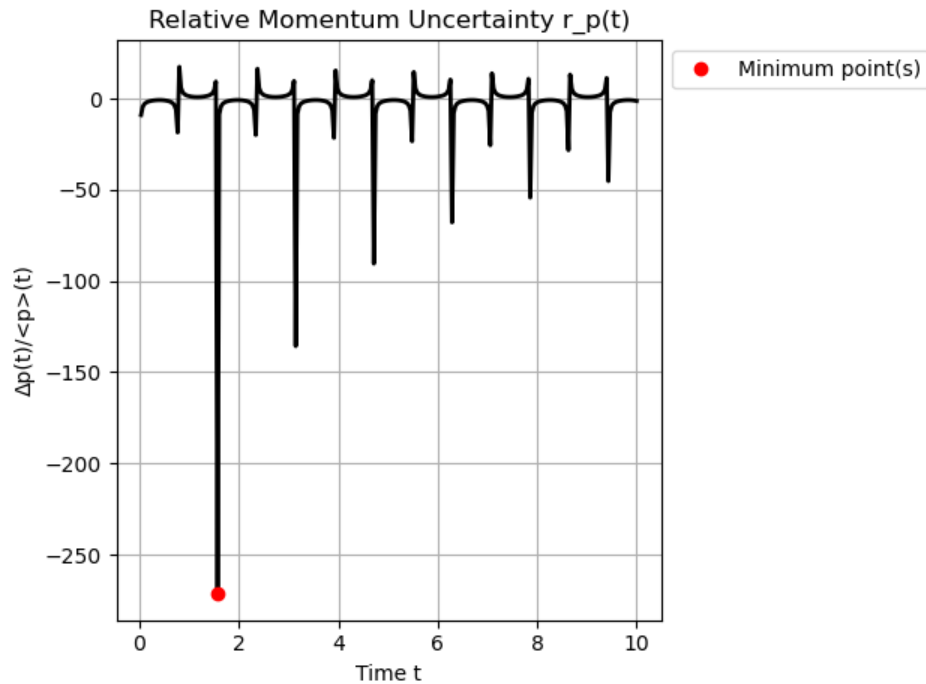
Saída Minimum value of $dp(t) / \langle p \rangle(t)$: -271.448595923341827

Estimated period (s):

$t = 1.571906354515050$

Full results saved in:

HalfOscillator/HalfOscillator_min_relative_momentum_uncertainty.txt



Por fim, para a validação dos resultados obtidos, verifica-se a conformidade dos valores calculados com o princípio da incerteza de Heisenberg. Nesse contexto, emprega-se o método `heisenberg_uncertainty()`, o qual retorna uma função dependente do tempo. A partir dessa função, são determinados valores específicos dentro do intervalo considerado, possibilitando a análise da consistência dos resultados em relação ao referido princípio. Procedimento executado:

Comando `heis = q.heisenberg_uncertainty(coefficients)`

```
# iteração para os primeiros dez valores inteiros
for i in range(0,11):
    print(f"t= {i} | {heis(i)}")
```

Saída

```
t= 0 | {'dx': np.float64(0.7045866278301305), 'dp':
np.float64(1.0917163361527396), 'dx.dp': np.float64(0.769208731836924),
'hbar/2': 0.5, 'valid': np.True_}
t= 1 | {'dx': np.float64(0.7743776899658102), 'dp':
np.float64(1.766205227286292), 'dx.dp': np.float64(1.3677099239114976),
'hbar/2': 0.5, 'valid': np.True_}
t= 2 | {'dx': np.float64(0.7660566197126013), 'dp':
np.float64(1.2357449111006344), 'dx.dp': np.float64(0.9466505694248011),
'hbar/2': 0.5, 'valid': np.True_}
```

```

t= 3 | {'dx': np.float64(0.6870972442011228), 'dp':
np.float64(1.1264243465585375), 'dx.dp': np.float64(0.7739630643214217),
'hbar/2': 0.5, 'valid': np.True_}
t= 4 | {'dx': np.float64(0.7041331979839964), 'dp':
np.float64(2.319239362707566), 'dx.dp': np.float64(1.633053429353644),
'hbar/2': 0.5, 'valid': np.True_}
t= 5 | {'dx': np.float64(0.6987535283048425), 'dp':
np.float64(1.1375393328295853), 'dx.dp': np.float64(0.7948596224002092),
'hbar/2': 0.5, 'valid': np.True_}
t= 6 | {'dx': np.float64(0.6971648426204864), 'dp':
np.float64(1.1379622136010508), 'dx.dp': np.float64(0.7933472475532368),
'hbar/2': 0.5, 'valid': np.True_}
t= 7 | {'dx': np.float64(0.7022178135873733), 'dp':
np.float64(2.3295360063571118), 'dx.dp': np.float64(1.6358416810571526),
'hbar/2': 0.5, 'valid': np.True_}
t= 8 | {'dx': np.float64(0.6864533638039139), 'dp':
np.float64(1.1278085414550556), 'dx.dp': np.float64(0.7741879670086087),
'hbar/2': 0.5, 'valid': np.True_}
t= 9 | {'dx': np.float64(0.7642232086943346), 'dp':
np.float64(1.226511862010146), 'dx.dp': np.float64(0.9373288306870567),
'hbar/2': 0.5, 'valid': np.True_}
t= 10 | {'dx': np.float64(0.7728906109430645), 'dp':
np.float64(1.7869350675343434), 'dx.dp': np.float64(1.381105336062205),
'hbar/2': 0.5, 'valid': np.True_}

```

Nesta última etapa, as soluções podem ser checadas e verificadas, de acordo com critérios previamente estabelecidos, utilizando o método:

Comando `q.check_solutions_numerically()`

Saída Building matrices for verification...
Reporting errors in equations in file:
HalfOscilattor/HalfOscilattor_Check_Equations.txt
Reporting errors in eigenvector normality in file:
HalfOscilattor/HalfOscilattor_Check_Normality.txt
Reporting errors in eigenvector orthogonality in file:
HalfOscilattor/HalfOscilattor_Check_Orthogonality.txt
Numerical verification Done.

Como mostrado na saída da execução do método anterior, os resultados são armazenados em três arquivos diferentes que armazenam os erros calculado para as equações, a normalidade e a ortogonalidade. Estes resultados podem ser visualizados graficamente utilizando o método

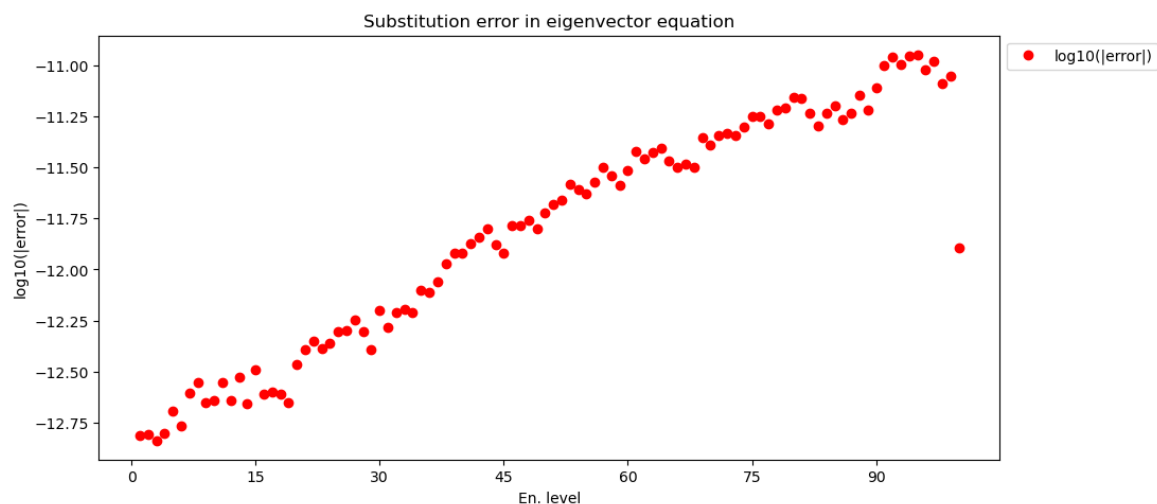
`check_solutions_graphically()`, que lê os resultados salvo nos arquivos e gera gráficos correspondentes. O procedimento pode ser executado seguindo a sintaxe:

Comando `p.check_solutions_graphically();`

Saída

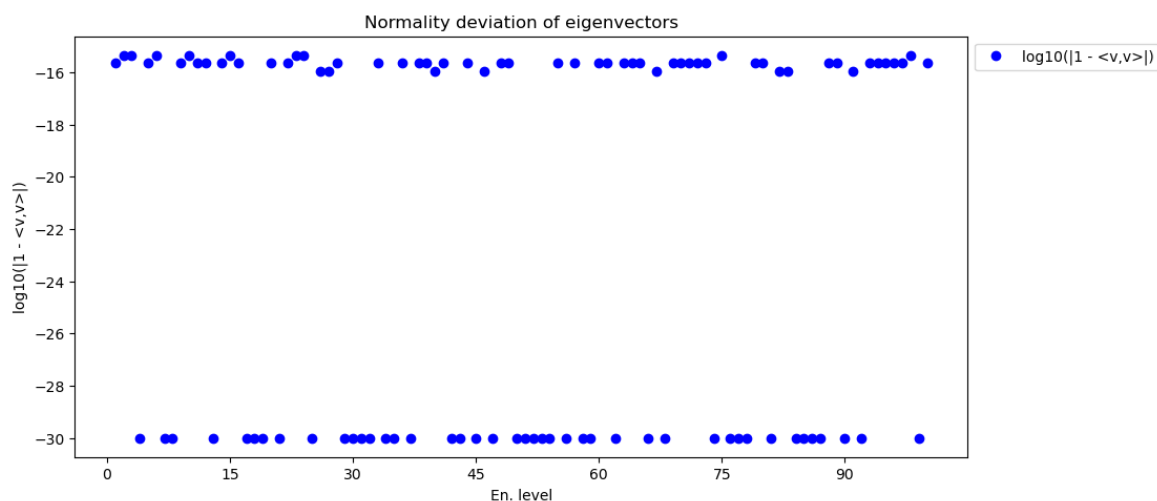
----- Substitution Error -----

Reading substitution errors HalfOscilattor_Check_Equations.txt

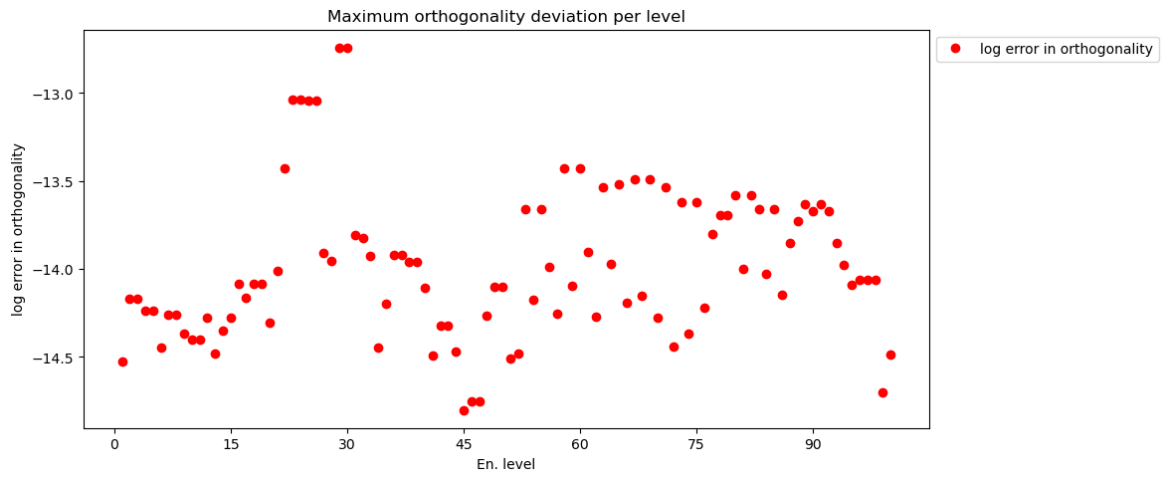


----- Normality Deviation -----

Reading deviations from normality of HalfOscilattor_Check_Normality.txt

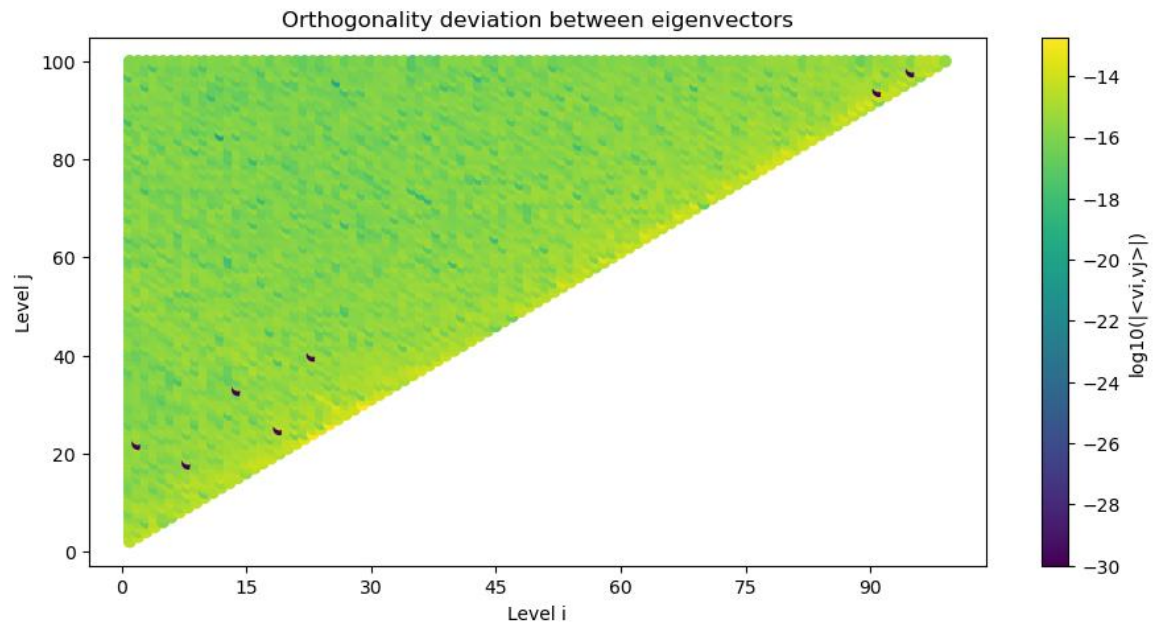


---- Creating summary plot of orthogonality deviation (max per level) ----



----- Orthogonality Summary (2D curve) -----

Reading orthogonality deviations from
HalfOscilattor_Check_Orthogonality.txt



No gráfico `substitution erros` observa-se que, conforme o nível de energia aumenta, o erro tende a crescer de forma geral, indicando maior dificuldade numérica na obtenção de soluções precisas para estados mais excitados. No entanto, há uma queda abrupta no último ponto, sugerindo que, para aquele nível específico, o método empregado conseguiu uma aproximação mais estável ou que ocorreu uma particularidade na estrutura matemática do problema.

No segundo gráfico, `Normality Deviation`, nota-se que os valores permanecem extremamente baixos (em torno de 10^{-30} , no nosso método representa erro zero), o que indica que os autovetores estão muito bem normalizados e que o método numérico utilizado é altamente preciso nesse aspecto. Pequenas flutuações ao longo dos níveis de energia não comprometem a qualidade geral, reforçando a confiabilidade das soluções obtidas.

No terceiro gráfico é representado o desvio máximo de ortogonalidade dos autovetores em função do nível de energia. Os valores se mantêm em torno de 10^{-12} a 10^{-15} , indicando que os autovetores permanecem

fortemente ortogonais, com erros muito pequenos. Essa estabilidade ao longo dos diferentes níveis de energia reforça a confiabilidade do método numérico utilizado, já que a ortogonalidade é uma propriedade essencial para a consistência das soluções em problemas quânticos. Por fim, o último gráfico apresenta o desvio de ortogonalidade entre pares de autovetores, representado em forma de mapa de calor. Observa-se que os valores permanecem muito baixos (entre aproximadamente 10^{-30} a 10^{-13}), o que indica que os autovetores são fortemente ortogonais e que o método numérico mantém alta precisão. As variações de cor refletem pequenas diferenças entre pares de níveis, mas sem comprometer a propriedade essencial de ortogonalidade, reforçando a consistência global das soluções.

The Wheeler-DeWitt equation

A quantização de um modelo cosmológico FRW para um fluido de matéria rígida e constante cosmológica negativa pode ser realizada a partir do hamiltoniano clássico [7]:

$$H = -\frac{p_a^2}{12a} + 3a + \Lambda a^3 + p_T/a^3$$

em que Λ é a constante cosmológica, a é o fator de escala do Universo, T é a variável canônica associada ao fluido de matéria rígida, e (p_a, p_T) são os momentos canonicamente conjugados a (a, T) , respectivamente. O formalismo variacional de Schutz [8] [9] produz a equação de Wheeler-DeWitt:

$$\left[-\frac{1}{12} \frac{\partial^2}{\partial a^2} - (3a^2 + \Lambda a^4) \right] \psi(a, \tau) = i \frac{1}{a^2} \frac{\partial}{\partial \tau} \psi(a, \tau)$$

Em que $\psi(a, \tau)$ é a função de onda do Universo e $\tau = -T$ desempenha o papel de “tempo” neste formalismo. A separação das variáveis produz uma solução trivial para a parte temporal, levando a $\psi(a, \tau) = \eta(a)e^{-iE\tau}$. Assim, a equação independente do tempo será:

$$\left[-\frac{\partial^2}{\partial a^2} - (36a^2 + 12\Lambda a^4) \right] \eta(a) = i \frac{12}{a^2} E \eta(a)$$

O que corresponde a fazer $f(a) = -(36a^2 + 12\Lambda a^4)$ e $g(a) = \frac{12}{a^2}$ na equação geral para o método espectral.

Além disso, o operador hamiltoniano fica:

$$H = -\frac{\partial^2}{\partial a^2} - (36a^2 + 12\Lambda a^4)$$

Sendo auto adjunto em relação ao produto interno para a função peso:

$$w(a) = \frac{1}{a^2}$$

Na ref [7], é usado $\Lambda = -0.1$, $N = 100$ níveis de energia e $L = 6$ como a amplitude do intervalo para os cálculos. Vamos definir o problema e atribuí-lo à variável p , associando-lhe o rótulo “StiffMatter”.

Utilizando a biblioteca `PySpectral`, o problema pode ser definido pelo comando:

```
Comando # a função 'f' define o potencial do problema.
def f(x):
    return -36*(x**2)+1.2*(x**4)

# a função 'g'.
def g(x):
    return 12/(x**2)

# a função 'w' define a função peso do problema.
def w(x):
    return 1/(x**2)

# definindo o objeto
```

```
p = SpectralMethod(num_levels=100, length=10, f_function=f, g_function=g,
weight=w, num_digits=15, label="StiffMatter", root_filename='StiffMatter',
optimizer_L='n')

p.is_described()
```

Saída -----

```
Number of energy levels:      100
Interval amplitude (Length):  10.0 (float: 10.0)
f function (V_spectral):      -36*(x**2)+1.2*(x**4)
g function:                   12/(x**2)
w function (weigth):          1/(x**2)
Label:                        StiffMatter
Number of digits to be used:  15
The energy spectrum has not been calculated yet.
-----

Important: when using the spectral method, the potential must be expressed
as V_spectral = 2*m*V_real.
```

Dessa maneira, criamos uma instancia `p` da classe espectral, podendo utilizar os métodos disponíveis para a classe. A solução pode ser calculada através da execução do seguinte método:

Comando `p.is_solved()`

Saída Eigenvalues already calculated. Skipping eigenvalue calculation.
Calculating eigenvectors ...

```
Calculating energy eigenvectors now ...
Eigenvectors saved to: StiffMatter_Eigenvectors.txt
```

```
Done!

=====
Time elapsed for eigenvector calculation: 17.049726 seconds.
=====

The eigenvalue/eigenvector problem has been completely solved with 15
digits used.
```

Em seguida, podemos verificar, por exemplo, os 10 autovalores de energia mais baixa do sistema, através do comando:

Comando `p.its_eigenvalues(slice(1, 11))`


```
Saída [np.float64(-380.22012695022),
        np.float64(-342.1147708538197),
        np.float64(-305.91471592040614),
        np.float64(-271.6318932477189),
        np.float64(-239.27917734798697),
        np.float64(-208.87051288608345),
        np.float64(-180.4210664335063),
        np.float64(-153.94741001279823),
        np.float64(-129.4677456328548),
        np.float64(-107.00218356354394)]
```

O valor esperado do fator de escala $\langle a(t) \rangle$ pode ser obtido através do método `expected_position`. Nesse caso, é necessário passar uma lista com os coeficientes que representam a combinação linear da autofunção. Considerando uma superposição equiprovável dos três primeiros níveis, podemos definir os coeficientes de acordo com a sintaxe da linguagem Python e utilizar o método, de acordo com o exemplo:

```
Comando coefficients = [1,1,1]+[0]*97
          Av = p.expected_position(coefficients = coefficients)
```

Dessa forma, a variável, A_v passa a ser uma função do tempo t . Logo, o valor esperado para o fator de escala $\langle a(t) \rangle$ no tempo $t = 0$, pode ser obtido fazendo:

```
Comando Av(0)
```

```
Saída 4.686383523744297
```

A incerteza para o fator de escala em função do tempo é obtida através do método `position_uncertainty`:

```
Comando coefficients = [1,1,1]+[0]*97
          Un = p.position_uncertainty(coefficients)
```

Novamente, passamos os coeficientes para o método. Assim, a incerteza associada posição no instante de tempo $t = 0$ é calculada fazendo:

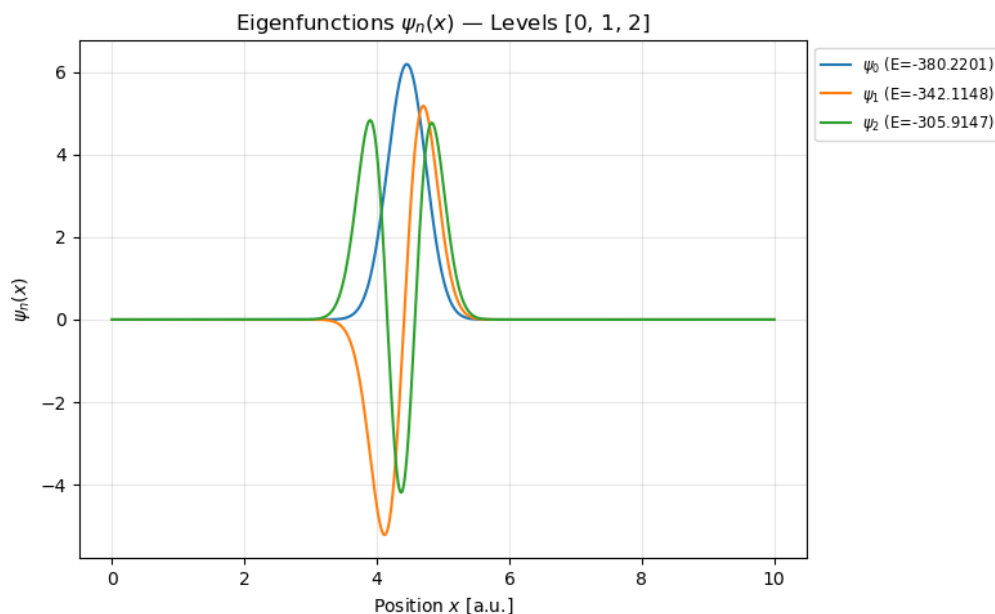
```
Comando Un(0)
```

Saída 0.2631601226187416

Para este exemplo, utilizando o método `p.plot_eigenfunctions(num_levels=3)`, foram gerados os gráficos para as funções de onda dos primeiros três níveis.

Comando `p.plot_eigenfunctions(num_levels=3)`

Saída Eigenpairs saved in: StiffMatter/StiffMatter_Eigenpairs.txt
--- Processing 3 levels---
-> Saved TXT: level 0
-> Saved TXT: level 1
-> Saved TXT: level 2
-> Image saved: StiffMatter/StiffMatter_eigenfunctions.png



Da mesma maneira que o exemplo da seção anterior, a unitariedade da dinâmica foi verificada por meio da conservação da norma da função de onda, utilizando o método `norm_of_wave_function`, responsável por calcular a norma da função de onda em função do tempo, permitindo avaliar seu comportamento em diferentes instantes e confirmar a consistência física da solução.

Comando `n = p.norm_of_wave_function(coefficients)`
`for t in np.linspace(0, 10, 11):`
`print(f"Norm at t= {t:.1f}: {n(t):.15}")`

Saída Norm at t= 0.0: 0.999999999999355
Norm at t= 1.0: 0.999999999999563
Norm at t= 2.0: 0.999999999999964

```

Norm at t= 3.0: 1.000000000000016
Norm at t= 4.0: 1.000000000000003
Norm at t= 5.0: 0.9999999999999831
Norm at t= 6.0: 0.9999999999999858
Norm at t= 7.0: 1.000000000000013
Norm at t= 8.0: 1.000000000000004
Norm at t= 9.0: 1.000000000000042
Norm at t= 10.0: 1.000000000000013

```

Para a análise dentro de um intervalo de tempo maior, o procedimento `expected_position_and_uncertainty_are_calculated` pode ser utilizado. Este método calcula simultaneamente a posição e a incerteza associada dentro do intervalo de tempo solicitado, salvando o resultado em um arquivo txt (para o exemplo, com o nome: `StiffMatter_PositionUncertainty.txt`). Por padrão, é calculado 300 pontos dentro do intervalo, mas esse valor pode ser alterado. No nosso exemplo, vamos considerar apenas 100 pontos. Após a execução do comando, um aviso indicará que o arquivo com os resultados foi criado e salvo:

```

Comando coefficients = [1,1,1]+[0]*97 # coeficientes
          dt = (0, 10) # intervalo de tempo desejado
          p.expected_position_and_uncertainty_are_calculated(t_range=dt,
          coefficients=coefficients, num_points=300)

```

```

Saída Expected position and uncertainty data stored in file:
        StiffMatter_PositionUncertainty.txt

```

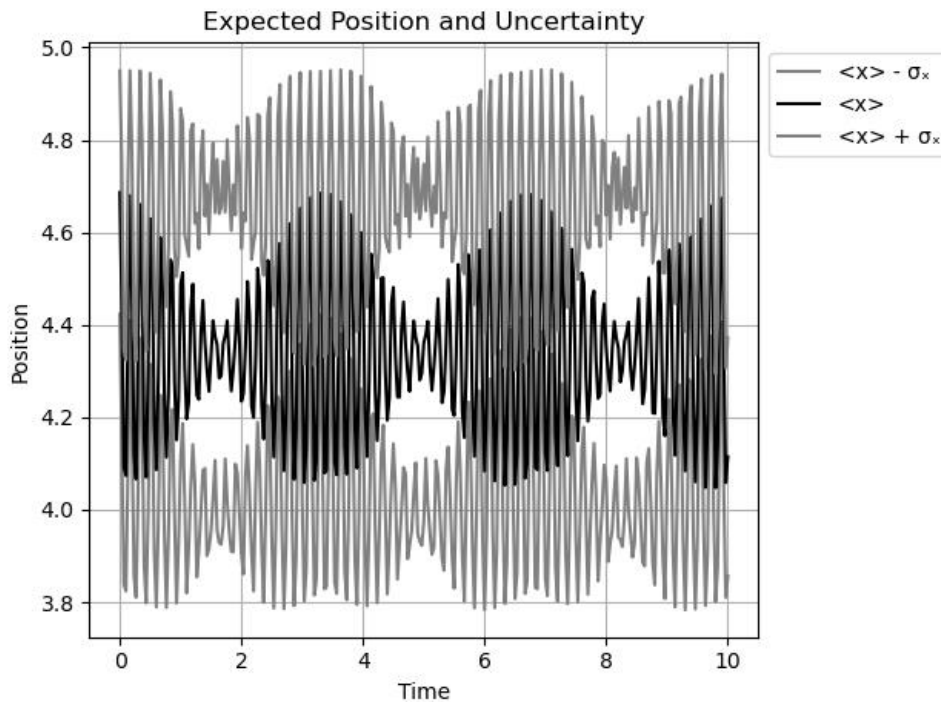
Realizado esse procedimento, os resultados podem ser visualizados graficamente. Para isso, o método `expected_position_and_uncertainty_are_plotted()` precisa ser executado. Este procedimento, lê os dados do arquivo txt (gerado pela função anterior) e gera um gráfico da posição com as incertezas, que é apresentado. Seu comando é dado por:

```

Comando p.expected_position_and_uncertainty_are_plotted( )

```

Saída



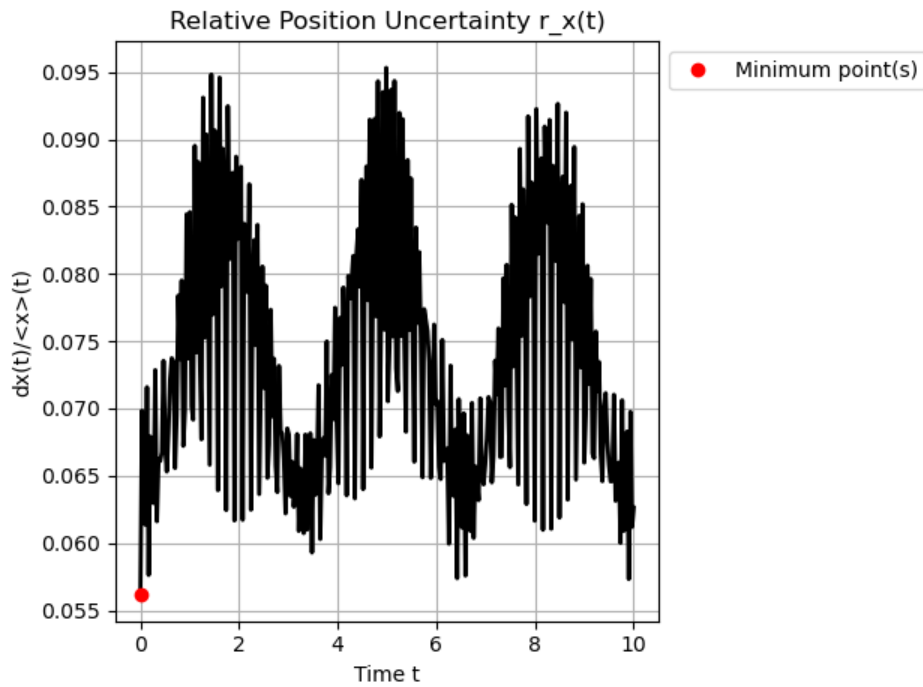
Podemos também analisar a incerteza relativa à posição em função do tempo. Para isso, executamos o comando `position_uncertainty_relative()`. Ele retorna, o menor valor encontrando no intervalo considerado, o período associado, quando houver tendência oscilatória nos dados e imprime o gráfico na tela. A seguir está o exemplo da execução desse comando:

Comando

```
tmax = 10
points = 300
p.position_uncertainty_relative(coefficients, t_max=tmax,
num_points=points)
```

Saída

```
Minimum value of dx(t) / <x>(t): 0.056154201056187
Estimated period(s):
t = 0.0000000000000000
Full results saved at:
StiffMatter/StiffMatter_min_relative_uncertainty.txt
```



Além dos dados obtidos para a posição esperada, é possível calcular e analisar o comportamento do momento esperado associado ao problema espectral em estudo. De maneira análoga, o momento pode ser determinado pelo método:

Comando `P_t = p.expected_momentum(coefficients=coefficients)`
`P_t(1)`

Saída 1.7993405875006094

O método `expected_momentum()`, retorna uma função. No exemplo apresentado, a variável `P_t` é definida como função do tempo. Dessa forma, para cada valor de tempo fornecido, a função retorna o momento correspondente ao respectivo instante. Além disso, é possível calcular a incerteza associada por meio da seguinte expressão:

Comando `dp = p.momentum_uncertainty(coefficients)`
`dp(1)`

Saída 3.5244764230480112

A partir deste ponto, torna-se possível calcular e gerar os gráficos referentes ao momento e às incertezas associadas. Para tal finalidade, emprega-se inicialmente o método `expected_momentum_and_uncertainty_are_calculated()`, responsável por realizar o cálculo do momento e das respectivas incertezas, armazenando os resultados em um arquivo de texto. A execução do método pode ser realizada conforme o seguinte procedimento:

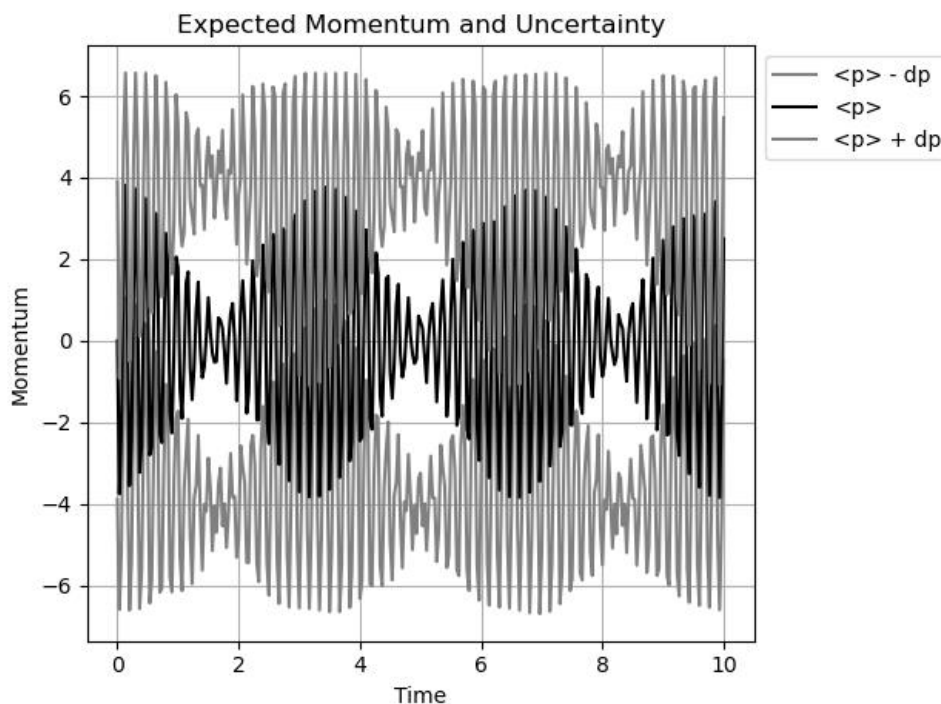
Comando `dt = (0.0, 10) # intervalo de tempo`
`points = 100 # número de pontos gerados`
`p.expected_momentum_and_uncertainty_are_calculated(dt, coefficients,`
`points)`

Saída Expected momentum and uncertainty data stored in file:
 StiffMatter_MomentumUncertainty.txt

A partir dos valores armazenados no arquivo de texto, torna-se possível visualizar graficamente os resultados obtidos por meio do método `expected_momentum_and_uncertainty_are_plotted()`. Esse procedimento permite representar tanto o comportamento do momento quanto as incertezas associadas ao problema em análise.

Comando `p.expected_momentum_and_uncertainty_are_plotted()`

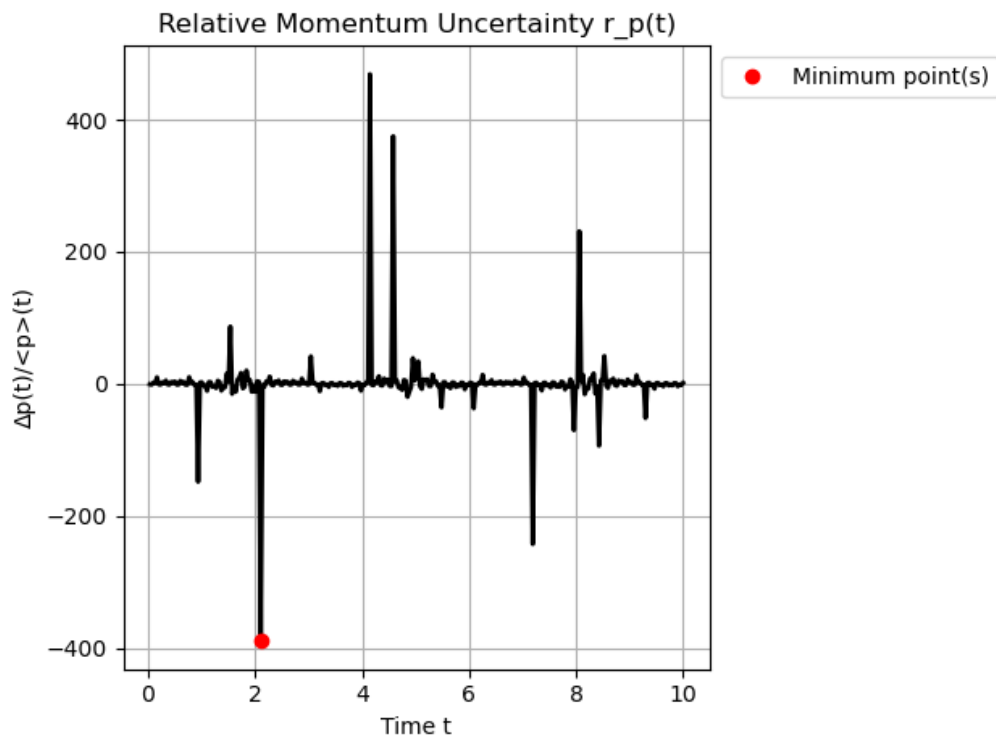
Saída



Para essa grandeza, é possível explorar a razão entre a incerteza e o momento. Nesse contexto, emprega-se o método `momentum_uncertainty_relative()`, o qual permite calcular essa relação de forma sistemática. A execução pode ser realizada conforme o procedimento a seguir:

Comando `tmax = 10`
`points = 300`
`p.momentum_uncertainty_relative(coefficients, t_max=tempo,`
`num_points=points)`

Saída Minimum value of $\Delta p(t) / \langle p \rangle(t)$: -389.317599998224409
 Estimated period (s):
 $t = 2.107023411371237$
 Full results saved in:
 StiffMatter/StiffMatter_min_relative_momentum_uncertainty.txt



Por fim, para a validação dos resultados obtidos, verifica-se a conformidade dos valores calculados com o princípio da incerteza de Heisenberg. Nesse contexto, emprega-se o método `heisenberg_uncertainty()`, o qual retorna uma função dependente do tempo. A partir dessa função, são determinados valores específicos dentro do intervalo considerado, possibilitando a análise da consistência dos resultados em relação ao referido princípio. Procedimento executado:

Comando `heis = p.heisenberg_uncertainty(coefficients)`

```
# iteração para os primeiros dez valores inteiros
for i in range(0,11):
    print(f"t= {i} | {heis(i)}")
```

Saída `t= 0 | {'dx': np.float64(0.2631601226187416), 'dp': np.float64(3.896164793380968), 'dx.dp': np.float64(1.0253152047689598), 'hbar/2': 0.5, 'valid': np.True_}`
`t= 1 | {'dx': np.float64(0.3797509967756505), 'dp': np.float64(3.5244764230480112), 'dx.dp': np.float64(1.3384234347647614), 'hbar/2': 0.5, 'valid': np.True_}`

```

t= 2 | {'dx': np.float64(0.3305749880338523), 'dp':
np.float64(4.220307584103972), 'dx.dp': np.float64(1.395128129114347),
'hbar/2': 0.5, 'valid': np.True_}
t= 3 | {'dx': np.float64(0.3119054557335758), 'dp':
np.float64(2.8530231062694242), 'dx.dp': np.float64(0.8898734721793868),
'hbar/2': 0.5, 'valid': np.True_}
t= 4 | {'dx': np.float64(0.2957678527581226), 'dp':
np.float64(3.8659583039270298), 'dx.dp': np.float64(1.143426186404931),
'hbar/2': 0.5, 'valid': np.True_}
t= 5 | {'dx': np.float64(0.38705582136215383), 'dp':
np.float64(3.7155360217800877), 'dx.dp': np.float64(1.4381198467107612),
'hbar/2': 0.5, 'valid': np.True_}
t= 6 | {'dx': np.float64(0.3000396726214886), 'dp':
np.float64(3.054320265415122), 'dx.dp': np.float64(0.9164172525163314),
'hbar/2': 0.5, 'valid': np.True_}
t= 7 | {'dx': np.float64(0.27397155148487823), 'dp':
np.float64(3.0506564838863284), 'dx.dp': np.float64(0.8357930899377408),
'hbar/2': 0.5, 'valid': np.True_}
t= 8 | {'dx': np.float64(0.2853514604685581), 'dp':
np.float64(4.624295434643282), 'dx.dp': np.float64(1.319549455913546),
'hbar/2': 0.5, 'valid': np.True_}
t= 9 | {'dx': np.float64(0.27890261748957274), 'dp':
np.float64(4.041346063906748), 'dx.dp': np.float64(1.127141995404774),
'hbar/2': 0.5, 'valid': np.True_}
t= 10 | {'dx': np.float64(0.2576664903599318), 'dp':
np.float64(2.978238157084891), 'dx.dp': np.float64(0.7673921733920951),
'hbar/2': 0.5, 'valid': np.True_}

```

Nesta última etapa, as soluções podem ser checadas e verificadas, de acordo com critérios previamente estabelecidos, utilizando o método:

Comando `p.check_solutions_numerically()`

Saída Reporting errors in equations in file:
StiffMatter/StiffMatter_Check_Equations.txt
Reporting errors in eigenvector normality in file:
StiffMatter/StiffMatter_Check_Normality.txt
Reporting errors in eigenvector orthogonality in file:
StiffMatter/StiffMatter_Check_Orthogonality.txt
Numerical verification Done.

Como mostrado na saída da execução do método anterior, os resultados são armazenados em três arquivos diferentes que armazenam os erros calculado para as equações, a normalidade e a ortogonalidade.

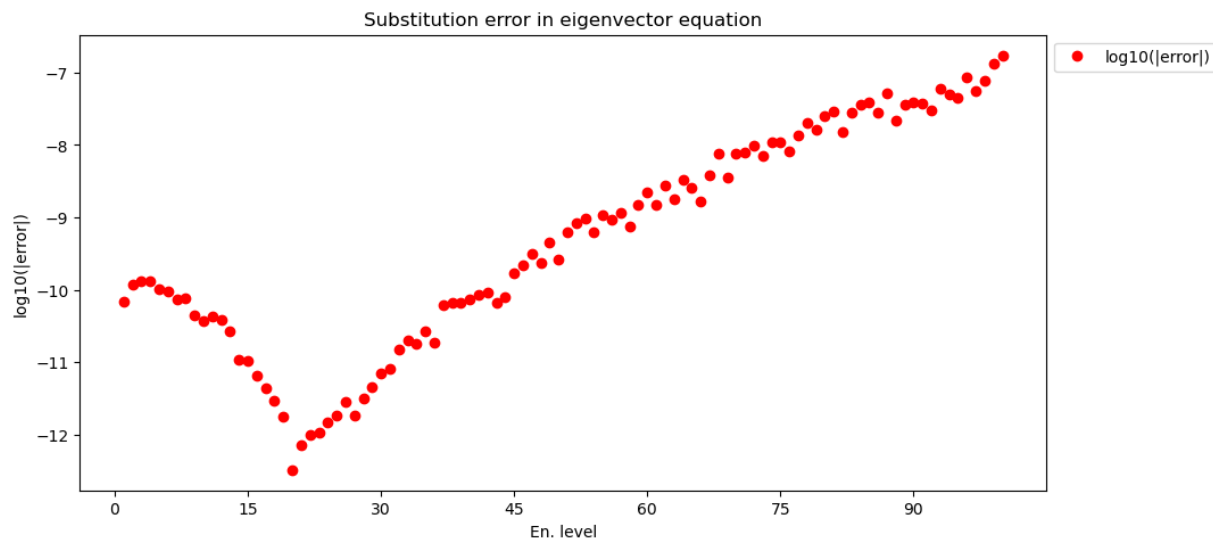
Estes resultados podem ser visualizados graficamente utilizando o método `check_solutions_graphically()`, que lê os resultados salvo nos arquivos e gera gráficos correspondentes. O procedimento pode ser executado seguindo a sintaxe:

Comando `p.check_solutions_graphically();`

Saída

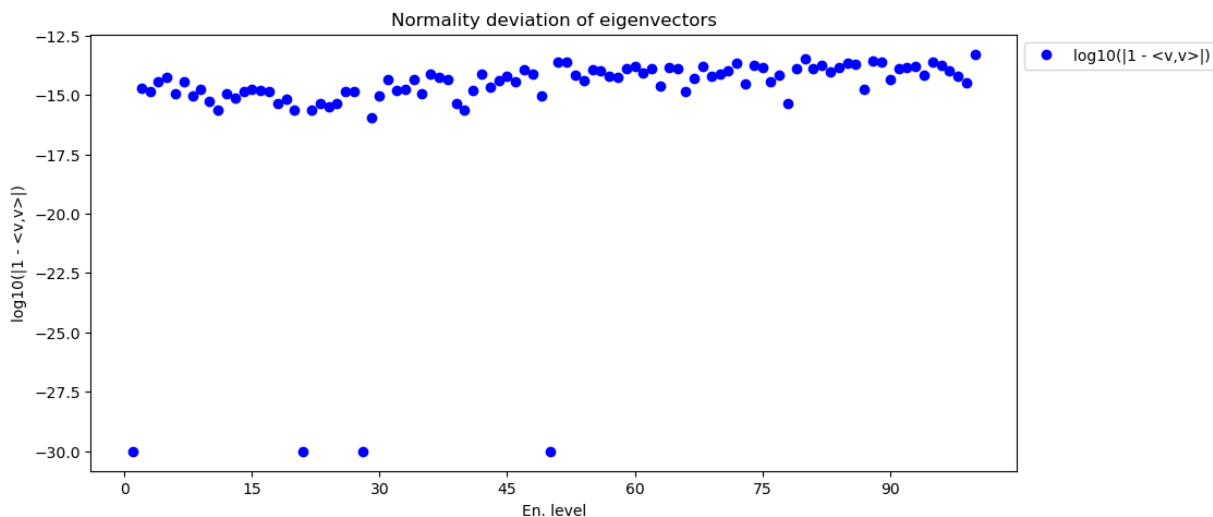
----- Substitution Error -----

Reading substitution errors StiffMatter_Check_Equations.txt

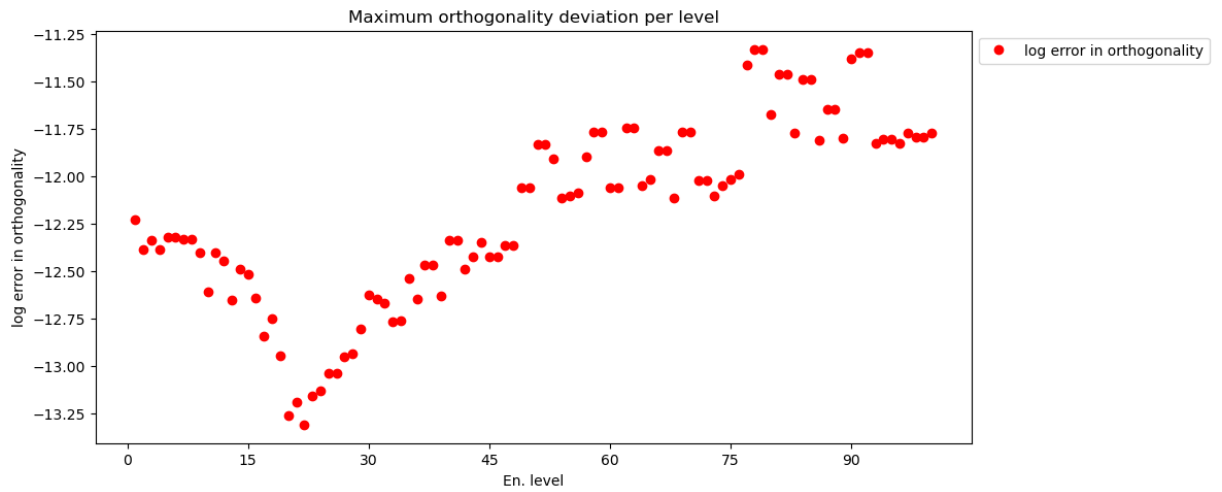


----- Normality Deviation -----

Reading deviations from normality of StiffMatter_Check_Normality.txt

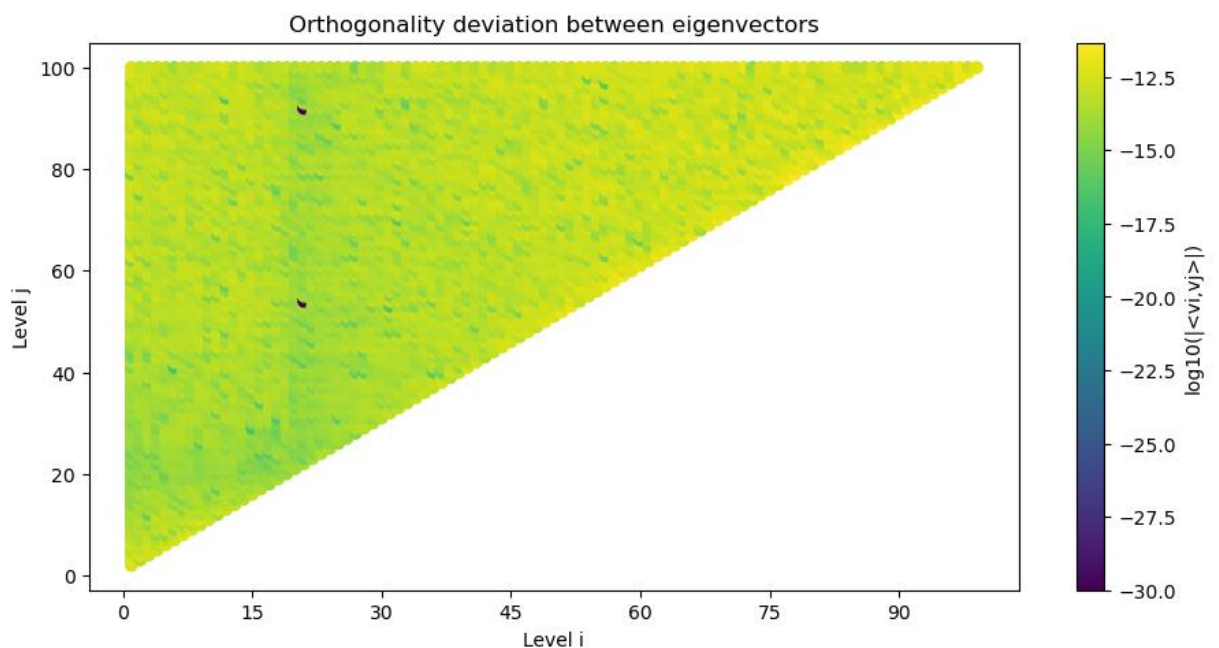


--- Creating summary plot of orthogonality deviation (max per level) -----



----- Orthogonality Summary (2D curve) -----

Reading orthogonality deviations from StiffMatter_Check_Orthogonality.txt



O primeiro grupo de dados (gráfico: `Substitution Error`) refere-se ao erro obtido ao substituir os autovalores e autovetores de volta na equação matricial de autovalores, com a mesma precisão utilizada nos cálculos. Os erros são exibidos em escala logarítmica. No exemplo mostrado, o erro varia de 10^{-13} a 10^{-6} , aproximadamente.

A segunda verificação está relacionada à normalização dos autovetores, levando em consideração a função peso (gráfico: `Normality Deviation`). Neste exemplo, o desvio da normalização varia de 10^{-16} a 10^{-12} , aproximadamente. Para um nível de energia específico, o desvio é inferior a 10^{-16} , indicado pelos pontos com valores 10^{-30} , considerados como nulos.

A verificação associada ao gráfico `Maximum orthogonality deviation per level`, mostra que, ao longo dos níveis de energia até $n=100$, no exemplo, o desvio máximo de ortogonalidade permanece baixo,

da ordem de 10^{-11} a 10^{-14} , o que confirma que os autovetores calculados são praticamente ortogonais dentro da precisão. A tendência de aumento do erro em níveis mais altos é esperada, já que estados excitados apresentam maior complexidade numérica, mas ainda assim os valores se mantêm em uma faixa aceitável para aplicações físicas.

A última verificação é a da ortogonalidade dos autovetores (gráfico: `Orthogonality deviation per level`). O i -ésimo autovetor ($i \in \{1, 2, 3, \dots\}$) é comparado com os j -ésimos autovetores ($j \in \{i + 1, i + 2, i + 3, \dots\}$). Aqui, mostramos a comparação do primeiro autovetor de menor energia com os demais, mostrando uma escala de gradiente para indicar a ordem de grandeza do desvio. O desvio da ortogonalidade varia de 10^{-30} a 10^{-12} , aproximadamente. Este gráfico evidencia que os autovetores calculados pela biblioteca PySpectral mantêm ortogonalidade numérica de alta qualidade. A predominância de valores baixos confirma que a base de autofunções é estável e confiável, mesmo quando se consideram pares de níveis elevados. As pequenas variações observadas refletem limitações inerentes ao cálculo numérico, sem comprometer a consistência global do método.

REFERÊNCIAS

- [1] Galerkin, Boris Grigoryevich. On the solution of differential equations of elastic equilibrium by means of series (Original em russo: Rods and Plates. Series in some problems of elastic equilibrium of rods and plates). **Vestnik Inzhenerov (Engineers Bulletin)**, 1, n. 19, 1915. 897-908.
- [2] ALVARENGA, F. G. et al. Quantum cosmological perfect fluid models. **General Relativity and Gravitation**, 34, n. 5, 2002. 651-663.
- [3] TAYLOR, R. L.; ZIENKIEWICZ, O. C. The finite element method. **Butterworth-Heinemann**, UK, 2013.
- [4] Monerat, G. A. *et al.* O método de Galerkin para a quantização de sistemas Hamiltonianos. **Revista Brasileira de Ensino de Física**, 37, n. 4, 2015. 4301-1.
- [5] Ballentine, Leslie E. **Quantum Mechanics: A Modern Development**. Singapore: World Scientific, 1998. ISBN 978-9810241056.
- [6] Griffiths, David J. **Introduction to Quantum Mechanics**. 1ª. ed. Upper Saddle River, NJ: Prentice Hall, 1995. ISBN 978-0131118928.
- [7] Oliveira Neto, G. *et al.* **International Journal of Modern Physics: Conference Series**, Singapura, 3, 2011. 254.
- [8] Schutz, B. F. Perfect Fluids in General Relativity. **Physical Review D**, Woodbury, 2, 1970. 2762.
- [9] Schutz, B. F. Hamiltonian Theory of a Relativistic Perfect Fluid. **Physical Review D**, Woodbury, 4, 1971. 3559.
- [10] SYMPY documentation. **Sympy**. Disponível em: <<https://docs.sympy.org/latest/index.html>>. Acesso em: 01 maio 2024.